Mars 2020 Project

Software Interface Specification (SIS)

Rover PLACES Data Products for PDS

Version: 1.0

2022-02-22

Prepared by:

Robert Deen, MIPL/IDS Data Scientist



Jet Propulsion Laboratory

California Institute of Technology

Pasadena, California

Copyright 2016 California Institute of Technology. Government sponsorship acknowledged.

1. Introduction 4
1.1. Quick-Start Guide 4
1.2. What is Localization 4
1.3. Primary Purpose of PLACES 5
1.4. PLACES database vs. PDS extraction 5
1.5. High-Level Requirements for PLACES 5
2. Background 7
2.1. Coordinate Frames and References 7
2.1.1. Rover (Nav) Frame 7
2.1.2. Site Frame 8
2.1.3. Local Level Frame 9
2.1.4. Orbital Frames 10
2.1.5. Reference Frames 10
2.2. Rover Motion Counter (RMC) 11
2.2.1. Site Index 11
2.2.2. Drive Index 11
2.2.3. Pose Index 12
2.2.4. Mechanism Indices 12
2.2.5. RMC as a Clock 12
2.2.6. RMC values of -1 12
2.3. Rover Behavior during Drives and Arm Moves 13
2.3.1. Driving 13
2.3.2. Arm Operations 13
2.4. Localization Methods 14
2.4.1. Onboard Methods 14
2.4.2. Ground-Based Methods 14
3. PLACES Localizations 16
3.1. Immutability of Results 16
3.2. Frames 16
3.3. Solutions 17
3.4. Views 17
3.5. Positions vs. Orientations 18
3.6. Metadata 18
3.7. Understanding the Derived-from Frame 18
3.7.1. Derived-from and Sites 20
3.7.2. Derived-from and Views 21
3.7.3. Derived-from and Orientations 21
3.7.4. Parent Views 21
3.8. Are Sites Parallel? 21
3.9. Mapping of Orbital Images 22
3.9.1. Orbital Image Map Projection 23
3.9.2. Projected Meters vs. True Meters 23
3.9.3. Elevation, Z, Areoid, and Radial Distance 23
3.10. Sources of Telemetry Data 24
4. PDS Bundle Contents and Structure 25
4.1. Sols Covered by a Delivery 25

4.2. Bundle Structure and File Types ... **25**
4.3. Versions ... **28**
4.4. Data Formats and Ranges ... **28**
5. Data File (Collection) Descriptions ... **30**
5.1. Collection: data_localization ... **30**
5.1.1. Included Views ... 30
5.1.2. View Metadata ... 32
5.1.3. Per-View CSV File ... 33
5.1.4. Raw Data For Each View ... 36
5.2. Colletion: data_maps ... **38**
5.2.1. m20_orbital_map.img, m20_orbital_map.xml ... 39
5.2.2. m20_orbital_dem.img, m20_orbital_map.xml ... 39
5.2.3. ODL Map Labels ... 39
5.2.4. PDS4 Map Labels ... 42
5.2.5. Using localization data in GIS programs ... 43
5.3. Collection: browse_maps ... **43**
5.4. Collection: data_orbital ... **43**
5.4.1. Metadata ... 44
5.4.2. Origin ... 46
5.4.3. Global Origin ... 46
5.5. Collection: data_mechanisms ... **47**
5.5.1. Sources of Information ... 47
5.5.2. Common Format ... 48
5.5.3. Mechanism Values and Meanings ... 49
5.5.4. Resolvers vs. Encoders ... 49
5.5.5. Arm ... 49
5.5.6. Drill ... 50
5.5.7. Drive ... 50
5.5.8. HGA ... 50
5.5.9. RSM ... 51
5.5.10. SHA ... 51
5.6. Collection: data_ancillary ... **51**
5.6.1. Observations table ... 52
5.6.2. SCLK Table ... 52
5.7. Collection: document ... **54**
6. Glossary ... **55**
7. Acronym List ... **58**
8. References ... **61**
9. Change Log ... **62**
Appendix A: Product Label Keyword Definitions ... **63**
Appendix B: Subset of PPPCS ... **65**

1. Introduction

This document describes the structure and use of data extracted from the PLACES (Position Localization and Attitude Correction Estimate Storage) database for the PDS (Planetary Data System) archive. These data cover primarily rover localizations and mechanism positions, along with a few ancillary items. The first few sections provide conceptual context; what is PLACES and how the data should be interpreted. Following sections contain the specific bundle, collection, and directory organizations, and file format specifications for the PDS delivery.

1.1. Quick-Start Guide

Most users of PLACES will be interested in just three files, along with their metadata labels:

- `best_interp.csv`, in the `data_localizations` collection
- `m20_orbital_map.img`, in the `data_maps` collection (or the .png in `browse_maps`)
- `m20_orbital_dem.img`, in the `data_maps` collection (or the .png in `browse_maps`)

The CSV file contains the location and orientation of the rover at each point along its traverse, which is the primary purpose of PLACES. The values are localized, meaning they have been adjusted so rover imagery matches the map, thus compensating for rover wheel slippage and other inaccuracies. The position is given in several ways, including latitude/longitude, global northing and easting in meters, and pixel coordinates on the map. The orientation is provided as a quaternion, and as the more convenient roll/pitch/yaw values. Locations are identified by Rover Motion Counter (RMC), i.e. site/drive/pose, as well as spacecraft clock and sol.

The two image files are a cutout from the base map of the entire rover traverse, plus ~500m on all sides. The pixel coordinates provide the location of the rover in these map images, which removes any ambiguity in how latitude/longitude are mapped. Both an image and a DEM (elevation values) are provided. Note there are also PNG versions of these images in the `browse_maps` collection, which might be more convenient to work with (but do not provide mapping information; see the label for the .img files themselves).

The rest of the files provide detailed information about a number of things, as described in this document. They are valid and users should feel free to make use of them if the information is useful. But for most users, who just want to know where the rover is at any given time, the three files above should be sufficient.

1.2. What is Localization

Rover localization is the process of determining the position (or location) and orientation (or attitude) of the rover at any given time. The true value is unknowable; only estimates (called "solutions") of the correct value exist. These estimates will differ depending on how they were produced and may sometimes conflict with each other. Yet each estimate is valid for its intended purpose and by its intended audience.

Localization solutions are produced both onboard the rover and by ground analysis of data.

1.3. Primary Purpose of PLACES

The primary purposes of PLACES are to manage all these solutions, from users throughout the operations and science teams as well as those produced by the rover itself, and to provide a coherent view of them across the Project.

Thus, PLACES is a "one-stop shopping" source for rover localizations. It applies no value judgments to the localizations; each is potentially as useful as any other. The user specifies which set of solutions to use (the View mechanism), and the database supplies those solutions in an easy-to-use manner.

1.4. PLACES database vs. PDS extraction

PLACES is an active database, accessed via ReST-style web services. This means that, in operations, each function is accessed via a URL and all queries and updates are atomic. All services of the database can be exercised using a simple Web browser, but are also available via command-line tools and programmatic requests. PLACES contains many active queries, for example performing on-the-fly coordinate frame transformations.

However, such a database is incompatible with the PDS archive. The purpose of PDS is to archive data in ways that will be accessible and usable decades if not centuries after the mission ends. It is unlikely that any active database will still be usable in that time frame, especially with no funding for maintenance. Therefore, for the PDS delivery, all the data have been extracted into static CSV (comma-separated-value) or XML files that are easily interpretable without special tools. The preparers of the archive have attempted to provide the data in as useful a form as possible, in some cases running active queries on the database to extract the output files. The active database query mechanism is not available in PDS.

The intent of this document is not to describe the active database; only the format and use of the extracted files is explicitly described. However, references to the database are common during explanations and examples. In general, "PLACES" refers to the data, the database, or both, as should be clear from context. The PDS bundle uses "Rover PLACES" to better disambiguate the term PLACES with other potential uses, but in this document generally just the name "PLACES" is used.

More information on the PLACES database itself, can be found in [2], which is an operations document, not part of the PDS archive. Although document is about MSL PLACES, the same principles apply to Mars 2020.

1.5. High-Level Requirements for PLACES

The requirements for the PLACES database are:

- Provide a single Project-wide source for localization solutions.
- Accept and store localization inputs from any team member.
- Provide access to all submitted raw localization data.

- Provide a simple and convenient mechanism to answer common queries.
- Provide views on the localization data that present a single position/orientation value for each Rover Motion Counter (RMC) instance.
- Guarantee immutability of results, so the same query always returns the same answer (subject to some restrictions).
- Provide access via Web service (ReST-style queries), Web page, programmatic request, and summary XML files.
- Store mechanism (joint angle) information as extracted from telemetry.
- Store other related ancillary data.

2. Background

This section provides some background on the Mars 2020 mission that is useful for understanding PLACES.

2.1. Coordinate Frames and References

A variety of coordinate frames are used on the Mars 2020 mission; there are literally dozens. Of these, however, only four are relevant to rover localizations, described below.

Note that PLACES works entirely in an XYZ Cartesian coordinate system, even for orbital data. It does not keep track of nor use latitude and longitude, although some products present conversions to latitude and longitude for convenience (see Section 5.1.3). See also the Camera SIS [1] for more on coordinate frames.

2.1.1. Rover (Nav) Frame

The Rover Navigation Frame (usually called just Rover frame in this document) is attached to the rover, with the origin fixed at a point on the nominal ground, between the middle wheels. This location is a virtual point that is defined assuming the rover is sitting flat and level, and does not change with respect to the rover body if the rover is tilted or the wheels articulate. If the rover is tilted, the origin of the frame is the same virtual point so it may not be exactly between the *actual* middle wheels. The +X axis points out the front of the rover, with +Z pointing away from the belly ("down"), and +Y completing the right-handed system. See Figure 1. Thus any part of the rover that doesn't move has a fixed coordinate in the Rover frame. However, the entire frame "moves" relative to the surface during drives, and thus the numeric coordinates of rocks as measured in a Rover frame change.

There therefore exist "instances" of the Rover frame, indexed by the RMC (see Section 2.2). Each instance is valid during a certain period of time, during which the coordinates of surface objects in Rover frame do not change.

The origin and orientation of these Rover frame instances describe the rover's location throughout time. Refining knowledge of these origins is the primary goal of rover localization.

Note that there also exists a Rover Mechanical frame, which is closely related to Rover Navigation but with a constant offset. It is not used in the PLACES system.

Because the Rover frame is attached to the rover, the position and attitude of the rover itself at any given time is the same as the position and attitude of the rover frame at the same time (assuming both are measured with respect to the same parent frame, which is normally Site frame). The concepts of rover pose (position + orientation) and Rover Frame pose are thus synonymous, and the terms are used interchangeably in this document.

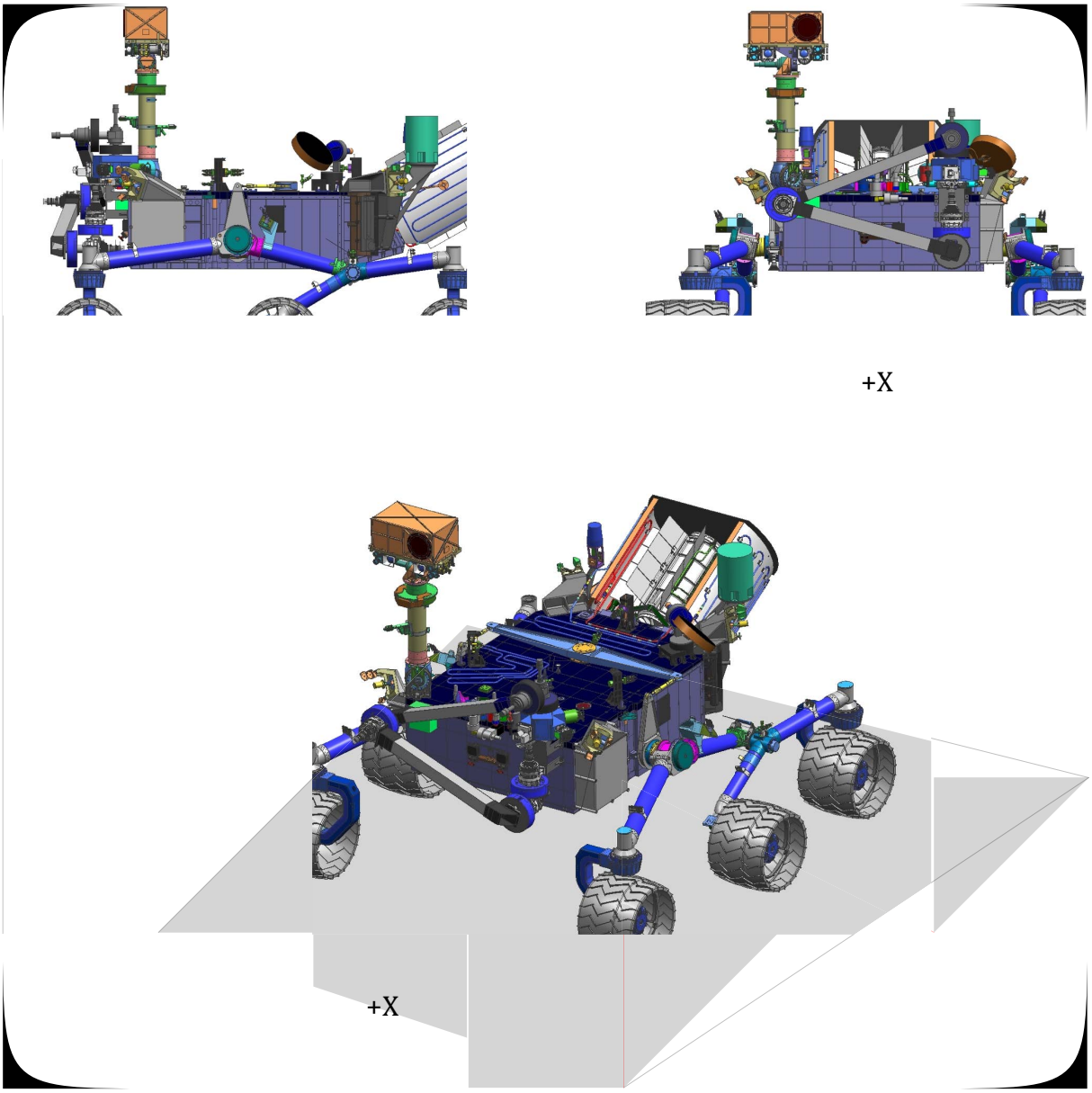


Figure 1: Rover navigation coordinate frame (RNAV)

2.1.2. Site Frame

Because the coordinates of surface objects measured in Rover frame change whenever the rover drives, it is advantageous to have a frame that is fixed to the surface.

However, on-board rover position estimation is imprecise; errors of 10% are common, more if the wheels slip excessively. Onboard Visual Odometry (visodom) reduces this error significantly, but does not eliminate it, and is not always used due to time and power constraints. If only one surface-fixed frame were available, accumulation of error over the mission would be significant. Therefore, multiple Site frames are used, each fixed to the surface at a certain place.

The Site frame thus defines the general vicinity of the rover. Most operations are carried out relative to the current Site frame. When the rover moves too far away from the current Site origin, a new Site is declared, with a new origin, and the Site index is incremented. Figure 2 illustrates this concept.

Sites allow isolation of localization errors. If it is determined that a particular drive was improperly localized early in the mission, only the origins of subsequent Site frames need to be corrected. All data within a subsequent Site are unaffected, because they are defined relative to their own Site origin.

Site frames are defined with +X pointing north, +Z pointing down according to the local gravity vector, and +Y completing the right-handed-system (east). The origin, by definition, coincides with the Rover (Nav) frame at the time the site is declared (and thus may not be on the actual ground if the rover wheels are articulated due to tilt).

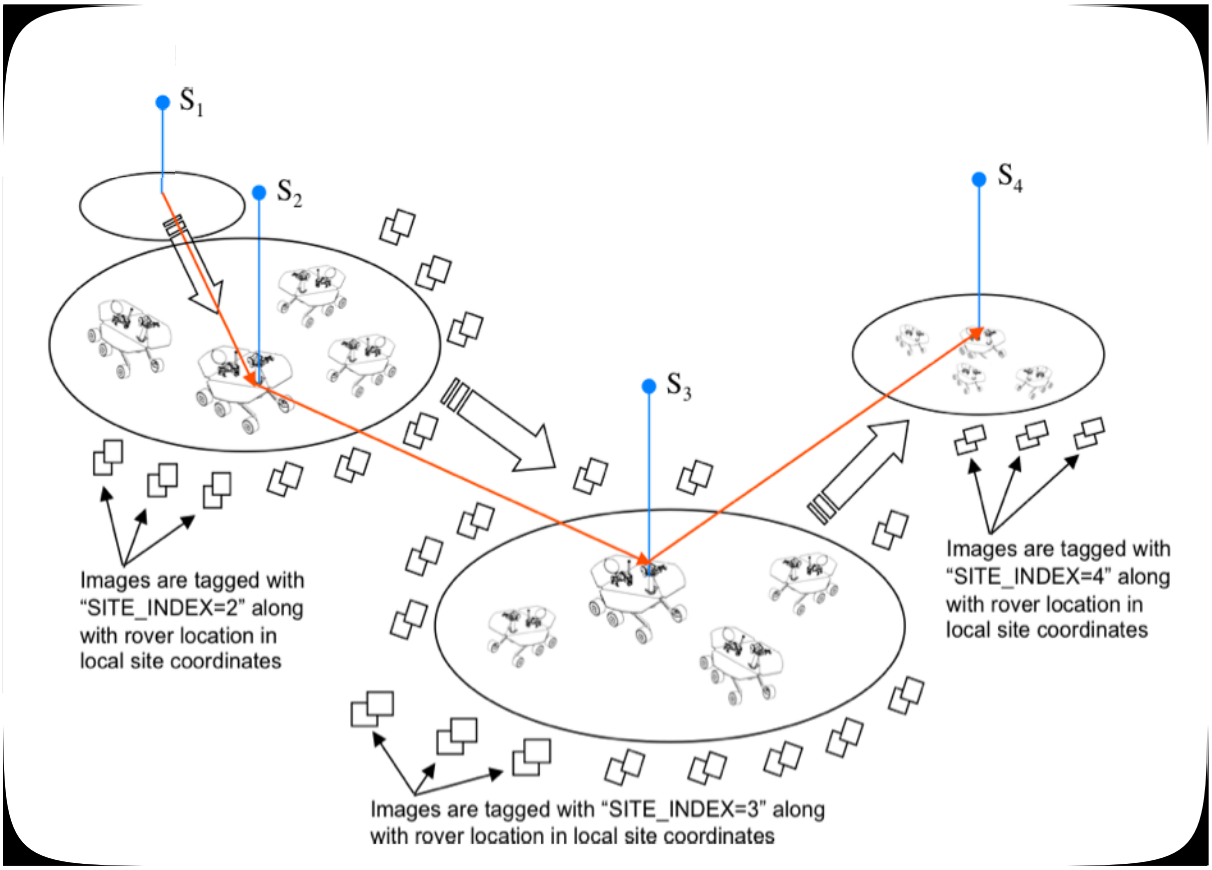


Figure 2: Site and Rover frame examples

2.1.3. Local Level Frame

Local Level frame is a combination of Site and Rover frames. It shares an origin with the Rover frame, thus moving with the rover and being instanced via the RMC. However, it shares the orientation of the Site frame, pointing north/down. The Local Level frame can always be easily computed given both the Site and Rover frames, therefore no explicit information about Local

Level frames is stored in the PLACES database. However, Derived-from frames (see Section 3.7) are commonly expressed in Local Level.

2.1.4. Orbital Frames

The final relevant frame type is Orbital frames. These are defined by orbital imagery, which must be in a rectangular projection. The details of orbital mapping are described in Section 3.9, but coordinates are defined in the same XYZ system as for Site frames (see below). Orbital frames are indexed in order to support more than one image or map. The ORBITAL(0) frame is special in that it defines a global XYZ coordinate system whose origin is the same as the cartographic latitude/longitude origin as defined by MOLA.

Important note: PLACES has its genesis in rover operations. Therefore, coordinate systems are defined in ways convenient to operations, with +X pointing north ("northing"), +Y pointing east ("easting"), and +Z pointing ***down*** (the *opposite* of elevation). This definition may be extremely surprising to cartographers. These XYZ coordinates define a Cartesian frame for local rover operations.

When applied to orbital frames, planet curvature must be taken into account. This is done by unrolling the planet into a cylindrical map projection. Thus, orbital frame XYZ coordinates are not body-centered physical 3-dimensional coordinates, but are rather coordinates in this unrolled (projected) map. So, X becomes the distance in meters north of some reference along the nominal, spherical surface; Y becomes the distance in meters (but see Section 3.9.2) east of some reference along the nominal surface, and Z becomes the distance below some reference. In the special case of ORBITAL(0), the reference is the equator, prime meridian, and datum elevation; thus X defines northing, Y defines easting, and Z defines distance below the datum. Other orbital frames specify their origin (reference point) in their metadata (see Sections 5.4.2 and 5.4.3).

Throughout this document and associated data, the term "Z" consistently refers to positive-down, while the term "elevation" consistently refers to positive-up. A simple sign change converts between the two.

Although more orbital frames may be defined over time, those available as of this writing are:

Orbital(0) is the global frame, defining northing and easting (see Section 3.9).

Orbital(1) is the base map (see Section 5.2.1).

Orbital(2) is the DEM that goes along with the orbital(1) base map.

Orbital(3) is a mosaic made using the Rover Descent Camera images (the mosaic itself is not delivered as part of Rover PLACES, but the frame defining it is included.

2.1.5. Reference Frames

All numeric values, whether a position or orientation, must be expressed relative to some other frame. It makes no sense to say the rover is at (10.5, 5.2, 0.1) without also saying how to interpret

those numbers. Thus, every coordinate or rotation must have a Reference frame, in which the values are expressed.

Generally, Rover frames are expressed in the Site frame sharing the same Site index. Site frames are generally expressed in terms of the preceding Site frame. However, these guidelines are not absolute, and any frame can be expressed in terms of any other, as long as a tree structure is maintained (i.e., no circular references). Ground-derived localizations, for example, are often expressed in the global (ORBITAL(0)) frame.

The root of the tree for PLACES is the Root frame, which is defined as the global origin (same as ORBITAL(0)). This frame has no numeric location, no solutions, exists implicitly in every view, and is identical across all views. Orbital frames and the landing site (Site 1 for Mars 2020 and MSL, Site 0 for MER) are all defined in terms of Root. Orbital(0) has a 0 offset and identity quaternion from root.

2.2. Rover Motion Counter (RMC)

The Rover Motion Counter (RMC) is a set of ten indices whose values serve to uniquely identify every location the rover has visited, and thus every instance of the Rover Frame. It also serves to indicate when any mechanism on the vehicle has moved. In many cases an abbreviated form of RMC is used, most commonly just (Site, Drive). Such abbreviations drop trailing elements, so the order is always the same.

The ten indices, in order, are:

Site, Drive, Pose, Arm, SHA, Drill, RSM, HGA, BITCAR, Seal

Note that the PIXL instrument uses two additional RMC elements (RTT, PMC), but these are not used or tracked in PLACES.

2.2.1. Site Index

The Site index defines which instance of the Site frame is relevant for this RMC. Whenever the Site frame is incremented, all the other RMC values are set to 0. Declaring a new Site frame thus resets all motions and creates a new local area in which to work.

Unlike the other indices, there is no meaning to odd or even values. Also, unlike other indices, Sites start with 1 (the landing site), not 0.

2.2.2. Drive Index

The Drive index (sometimes incorrectly called Position) increments whenever the rover drives or otherwise moves its wheels (e.g., trenching or steering). The value is odd while the wheels are actually moving, and even while they are not. Incrementing Drive sets all the subsequent indices (i.e., all except Site) to 0. Exception: if a mechanism index is odd, indicating it is moving during the drive, then that index is not reset to 0. This should be an unusual case, however. The Drive index is the one of higest interest for most localization activities.

2.2.3. Pose Index

The Pose index indicates a change in the rover's knowledge of its position or orientation, when it is not actually driving. Unlike the other indices, it does not indicate that anything actually moved; only that the pose knowledge has changed. This may be due to running visodom (visual odometry), doing a sun find, reading the IMUs (Inertial Measurement Units), or receiving an explicit ground command to update the pose knowledge. Note that the rover might actually have moved slightly, mostly in orientation, due to being pushed by the arm (e.g., during a drill preload). There is unfortunately no reliable way to distinguish this from an attitude update, although the arm index will change during arm movements. So, a pose change does not indicate arm-induced motion, but lack of an arm update does rule it out.

Pose will be odd while an update is being made (including extended IMU updates), even otherwise. Incrementing Pose has no effect on other indices.

2.2.4. Mechanism Indices

The remaining RMC indices (Arm, SHA, Drill, RSM, HGA, BITCAR, Seal) are associated with mechanisms on the rover. They increment to an odd number when the mechanism is moving, and again to an even number once the mechanism stops. More than one mechanism may be moving at the same time. More information about these indices can be found in Section 5.5

The Mars 2020 arm has the strength and mass to move the rover (in tilt, primarily) when it is used. Thus, the fact that the arm moved, as reflected in the RMC, means that the rover's physical pose may have changed, even if its knowledge has not. This is sometimes relevant and sometimes not, but is the primary reason for maintaining the mechanism RMC indices. The other mechanisms could in theory similarly move the rover, but are unlikely to in practice.

2.2.5. RMC as a Clock

The RMC can be thought of as a clock, albeit one that ticks at irregular intervals. It measures time not as a duration, but in terms of motions.

Given any two valid RMCs, one can determine unambiguously which occurred earlier in time, because the counters only count upwards (except when reset to 0 by incrementing Site or Drive). If the Sites are different, the lowest is earlier. If the same, then the lower Drive is earlier. If those are the same, then consider the remaining indices. If any are lower, then the entire RMC is earlier. Note that it is not valid for some of these indices to be lower and others higher - that indicates the RMC value was corrupted, possibly during transmission.

2.2.6. RMC values of -1

In the PDS archive data, the value -1 sometimes appears for an RMC index. This indicates "unknown" or "does not matter". For example, the Pose RMC index is usually -1 for localization solutions, because the Pose RMC index is meaningless for localizations. Mechanism tables often have -1 in RMC slots other than the mechanism of interest, as the other mechanisms are not relevant or unavailable. Specifically, the data products that contain the mechanism information often provide just the RMC index for that mechanism, and not the full RMC.

The value -1 never appears as part of a full RMC, thus it is available for use as a wildcard flag value by PLACES. It only indicates that the corresponding value is either unknown, or does not matter (or both).

2.3. Rover Behavior during Drives and Arm Moves

The interaction of the RMC with driving and arm motion is relevant to localization, so it is described here.

2.3.1. Driving

Before a drive starts, the Drive index is incremented to an odd value, where it stays throughout the drive. Once the drive ends, the index is incremented to even. During the drive, the rover position and orientation are computed and propagated by the flight software at an 8Hz rate. This means that the same RMC, with an odd-valued Drive index, can refer to many different places along the drive.

For this reason, odd-drive RMCs (called dynamic RMCs by PLACES) are insufficient to specify a location; the SCLK time must be used instead. However, very few other activities that require localization (such as imaging) happen during a drive, so this is rarely an issue. To handle the instances where it is (such as taking video while driving), PLACES is supposed to store all pose updates it sees in telemetry during a drive, for use with time-based queries. However, currently this is not the case; dynamic (odd-drive) RMC values are not regularly stored. They may be seen occasionally however, as reported in some data product.

Note that the Pose index does not increment during the drive, despite the fact that pose knowledge is changing.

If visual odometry is run to correct the rover's position after a drive, the drive index is first incremented to an even number. Pictures are then acquired and analyzed onboard the rover. Once a visodom solution is computed, the Pose index is incremented to an odd number, the value is stored, and Pose is incremented to even again. If the visodom image is downlinked, it will contain the position computed at the end of the drive, not the visodom solution.

Mars 2020 has an improvement in this area over MSL called "thinking while driving". In this mode, the rover can analyze images from location A while actively driving to location B. The solution is then back-propagated to the A frame by the flight software, where it shows up at a new pose for location A. It ends up looking the same as MSL's method (pausing while thinking) in the database.

2.3.2. Arm Operations

Because the arm has the possibility of moving (tilting) the rover, special precautions may be taken during arm operations. If enabled, the Pose counter is first incremented to an odd number. Arm operations then commence. While Pose is odd, the rover attitude is continually monitored via the IMU and updated at an 8Hz rate. Once the arm motion stops, Pose is incremented to an even number and the updates stop. Note that the Arm index will have incremented as well due to its motion, along with potentially those of other mechanisms.

This is similar to the behavior during driving, and RMCs with odd Pose values are also called dynamic RMCs. However, arm operations often take longer, and there are more likely to be other activities occurring at the same time. Thus it is more likely to see an odd Pose value in other data, such as images. Also, the rover position is not updated during arm operations; only the orientation is. PLACES will again store orientations seen in telemetry during moves, for use with time-based queries (Note: as with drives, dynamic orientations are not currently stored on a regular basis).

Not all arm operations will monitor the pose. If not, attitude knowledge will not change (even if the rover moves), meaning the Pose index will not change.

2.4. Localization Methods

Localizations can come from many sources, using many methods. The following list is not comprehensive, but is intended to illustrate what is meant by localization. Note that all onboard methods increment either the Drive or Pose indices (none of the ground-based methods do).

2.4.1. Onboard Methods

- Wheel rotation integration. This is the primary method for the rover to compute its position onboard and happens during all drives.

- Visual odometry (visodom). This is a more accurate onboard method for position estimation, that analyzes images taken before and after a drive to compute how the rover moved.

- IMU. Onboard intertial measurement units allow continuous estimation of orientation.

- Sun find. A more accurate method of estimating orientation on board, that takes images of the sun to compute how the camera, and thus the rover, is oriented.

2.4.2. Ground-Based Methods

- Image analysis. This is the process of analyzing images taken before and after a drive (or series of drives) to refine the pose. Similar to visodom, but likely to be more accurate in ground processing.

- Target updates. An interactive process by which a predefined target is adjusted to match newer images, thus creating a pose update.

- Orbital localization. Determining where the rover is in an orbital image, either by seeing it directly or (more often) by triangulating from landmarks. Can also be used for orientation, although this is not commonly used.

- Radio tracking. The Navigation team can use UHF two-way Doppler tracking of the rover to estimate rover position in the Mars inertial system, which can be converted to the Mars body-fixed or orbital-image reference frames used by PLACES. This is generally only of use during the descent; other methods are more accurate during surface operations.

- Bundle adjustment. A process that gathers information such as tiepoints and runs a global minimization function to adjust many results simultaneously in order to obtain the best overall solution.

3. PLACES Localizations

This section covers the fundamental concepts of PLACES as they relate to localization and the PDS delivery.

3.1. Immutability of Results

One of the top-level requirements of PLACES is to guarantee immutability of results, except in certain defined cases. That means that an identical query will return the same answer throughout time. This can be important for tactical planning, in order that ongoing localization activity does not disrupt target definitions.

During operations, every query to PLACES involving coordinate transforms returns a flag that indicates whether or not the result is immutable. Metadata (for example, who created a solution) that do not affect any numeric results are *not* immutable and can be changed at any time. They are also nullable and can be filled in later or left out.

Note that in practice during operations, immutability has not been an issue; all views have remained open and thus mutable. It is mentioned, however, because some of the output contains a flag indicating the immutable status of the data.

3.2. Frames

A "frame" (coordinate frame) specifies a mechanism for locating points within a reference frame. Frames within PLACES are identified by a frame type and an instance identifier, which may be an RMC, time value, or index. Note that a frame is not the same as a solution. Frames unambiguously exist; it's their location and orientation (contained in a solution) that is uncertain.

PLACES defines five frame types:

- Site
- Rover
- Orbital
- Local_Level
- Root

All of these except root correspond to the coordinate frames described in Section 2.1. It is the combination of frame type and associated instance identifier (RMC, time, or index) that specifies what is being defined. For example, a Site frame requires an index, thus Site 5 is a frame. So are Rover (5,12,2,6,100,8,0,0,12,2), Rover at SCLK 410455612, Rover (5,12), and Orbital image 3. Local_Level frames are not included in the Rover PLACES archive.

The Root frame is used as the single reference from which all others are ultimately derived, and it corresponds to ORBITAL(0). See Section 2.1.5.

3.3. Solutions

Solutions are the currency of PLACES. Localization is not an exact science, and multiple estimates of the rover pose from multiple places can be simultaneously valid for different reasons. For example, different people may generate different tiepoints and thus localize differently, or different mechanisms may be used (like manual vs. automatic tiepointing), or there may be different goals, such as minimizing error in a mosaic vs. minimizing error in a terrain mesh (for which the constraints are very different). Each of these estimates is called a "solution".

A solution is a single estimate of position or orientation for a single frame, relative to some reference frame. Once a solution is defined, it may never be changed for any reason.

Each solution is named by a solution ID. Solution IDs may be shared among multiple solutions. One frame can have several solution IDs associated with it, and one solution ID can be used for multiple frames. But the pair (Frame, ID) names a specific solution and must be unique.

A solution is used to define the relationship between two frames - the one that's part of the solution name, and the one named by the Derived-from frame (not the reference frame; see Section 3.7). If the Derived-from frame moves relative to some third frame, the solution's frame may move as well relative to that third frame - but the relationship between the two is fixed. For example, if someone redefines a Site frame, the Rover frames inside it retain the same relationship to the Site but may have a different relationship to an earlier Site. In other words, solutions exist in isolation.

3.4. Views

Working with multiple, possibly contradictory, solutions for the same frame would be extremely difficult. In addition, in order to make use of the information contained in a solution, it must generally be placed into a broader context, containing relationships to other frames, and thus other solutions. One generally needs a solution ID for each frame of interest, which is not practical to specify directly.

For these reasons, PLACES has a concept of named Views. A View is simply a way of looking at the database that assigns one and only one solution to each frame. Which solution that is will depend on how a particular View is defined.

Thus, a View is simply a list that specifies which one solution ID to use for each frame. It is shorthand, relieving the operations user of the burden of specifying a huge list of solution IDs every time they want to use the database.

Views are generally defined in terms of a parent View, with changes applied. For example, if Joe likes Sally's View but disagrees with a couple of rover positions, Joe could create a new View with Sally's as a parent, and then redefine the positions he disagrees with.

Views are thus logically dense (defining a solution for every frame), but are physically sparse (containing just those solutions that differ from the parent).

Views can be created by any operations user, and users can control who is able to modify their view. However, all views are public - there is no restriction on anyone reading a View. The intent of the PDS delivery is to present all non-trivial views that have been created in operations (see Section 5.1.1).

3.5. Positions vs. Orientations

Positions (offsets in XYZ space) and orientations (attitude of the rover) are treated completely separately by PLACES. Within one View, a given frame can have one solution ID for the position and a different one for the orientation. Although both are needed for a full coordinate frame transform, they are very commonly derived separately, so a full separation makes it easier to work with them. This document often talks about positions for convenience, especially in terms of coordinate transforms. In most cases, the concept applies equally well to orientations (where an offset maps to a rotation and adding offsets maps to compositing rotations).

3.6. Metadata

Most resources have associated with them a set of metadata. These metadata include things like who created the resource, why it was created, a description of it, who can update it, etc. Use of these metadata is not mandatory but it is highly recommended for documentation. For example, a solution ID might be created with user="rgd", institution="JPL-MIPL", method="marsnav image analysis", and description="Compensate for wheel slippage during rover bump in the middle of the BigCrater panorama". Such metadata help users determine which solutions or views they want to use.

These metadata are dumped to CSV files as part of the PDS delivery (see Sections 5.1.2 and 5.4.1).

3.7. Understanding the Derived-from Frame

This section talks about positions/offsets for convenience; it applies equally well to orientations/rotations.

Localization solutions have to be derived from something. They specify a new offset or rotation relative to some other frame. This frame need not be the same as the Reference frame! In other words, the source of the localization (the Derived-from frame) does not have to match the way in which the numbers are expressed (the Reference frame). See Figure 3.

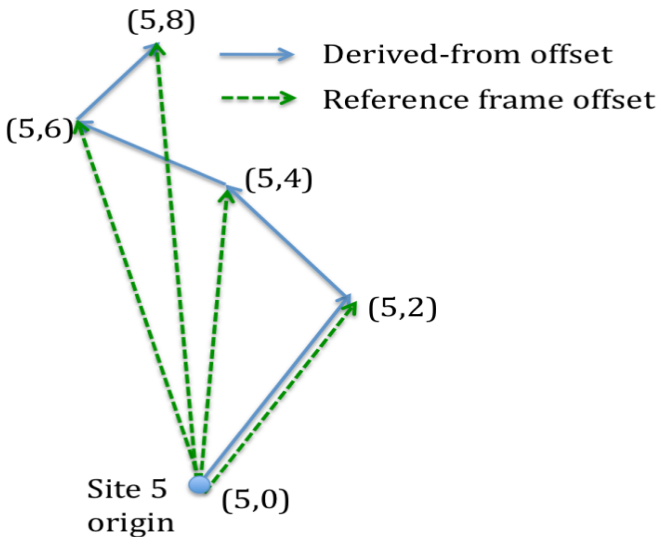


Figure 3: Derived-from and Reference frames for a drive sequence

The archetypal example of this is the case of the rover counting wheel rotations onboard. As the rover drives, it counts wheel rotations and integrates them to determine how far it went. All the rover can really determine is how far the *last* drive arc was. But the position is *expressed* in the Site frame. Consider the example in Figure 4. Let's say that for site 5, drive 12 is located at 5.0 meters (consider only the X coordinate for clarity). The distance from drive 12 to 14 was 2.0 meters, and from drive 14 to 16 was 1.0 meters. That means frame (5,14) has a derived-from frame of (5,12) with an offset of 2.0 meters, and a location of 7.0 meters relative to the reference frame, Site 5. Frame (5,16) is expressed as 8.0 meters from the reference Site 5, even though it's derived-from frame is (5,14) with an offset of 1.0 meters. Now say it is determined later that the drive from 12 to 14 was shorter than expected due to wheel slippage, with a distance of only 1.5 meters instead of 2.0. This is shown on the right side of Figure 4. The location as measured in the Site 5 frame for (5,14) is now 6.5. But the drive offset from 14 to 16 is still valid, at 1.0 meters. That means the position of (5,16) as measured in Site 4 should now be 7.5 meters instead of 8.0. The relationship between frame (5,16) and its derived-from frame (5,14) did not change - it's still 1.0 meters. But the relationship between (5,16) and its Reference frame (Site 5) did change.

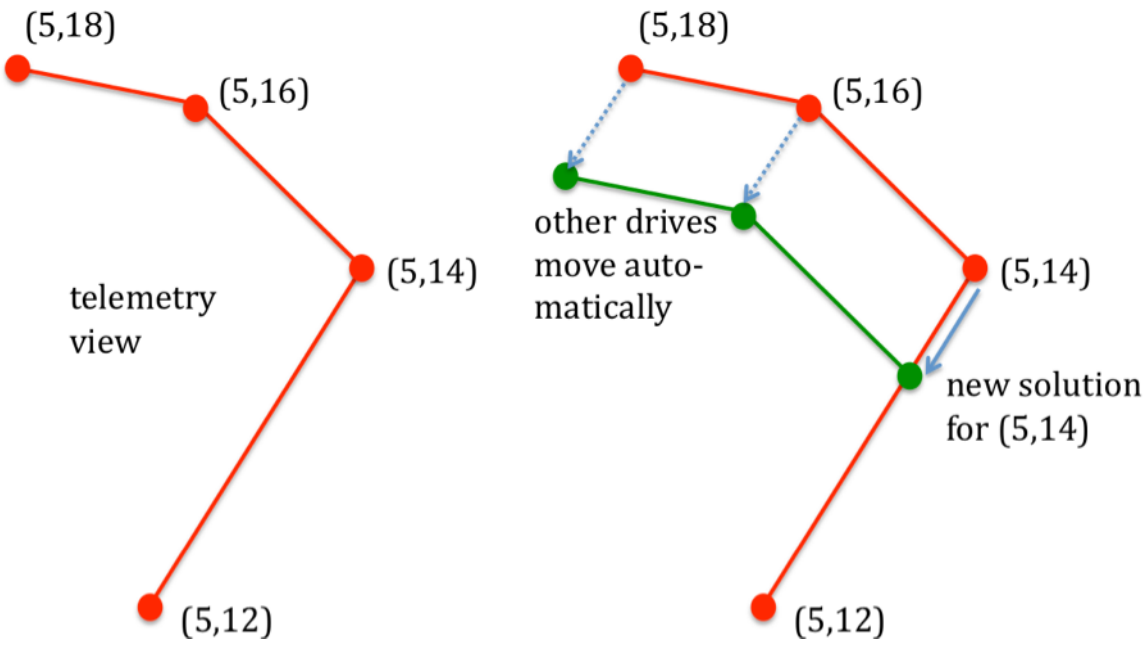


Figure 4: Result of moving one solution in a chain of Derived-from frames

This is the key point: for a given solution, the numeric relationship between the frame and its Derived-from frame does not change - even if the numbers are expressed in a different Reference frame. If the Derived-from frame moves, the solution frame derived from it moves along with it.

The active PLACES database keeps track of this and applies the appropriate corrections when queries are requested. In the PDS delivery, the Global Frame (Section 5.1.3) data have had these corrections applied. However, it is an important issue to understand if you want to make use of the Raw data (Section 5.1.4) directly.

3.7.1. Derived-from and Sites

When a Site frame is declared, the position of that site frame with respect to the previous frame is saved and downlinked as a site offset. These site offsets provide a link between each site and its predecessor.

But how is that site offset derived? It's derived as the position of the rover at the time the Site frame is declared. So, in reality, the Derived-from of a Site frame should be the last Rover frame before the Site was declared.

However, that is impractical for two reasons. First, the ultimate Rover frame at the time the site is declared may not (ever) be known, depending on how much data were telemetered. In the case of rover drives, we know the Derived-from is drive n-2; if n-2 is not present in the database, we can use the most recent prior drive with no loss of accuracy. However, in the case of a new site, we may not know the proper drive index to use - and picking the wrong one would lead to incorrect results.

A second factor is efficiency - it is not practical to check every single drive throughout the entire mission when doing coordinate transforms. Once the Derived-from chain gets to a site frame, site-to-site transforms can be used instead.

For these reasons, the Derived-from for a Site frame for "telemetry" is set to the previous Site frame (just as its Reference frame is).

The implication of this is that localizations within a site will not affect anything in the next site without explicit user action. This matches typical user expectations better, because sites are often considered to be independent realms.

3.7.2. Derived-from and Views

There is one other important factor to note regarding Derived-from and Reference frames: they must use the same View. This common View is attached to the Reference frame by convention, but it applies equally to both frames. As described above, solutions are ultimately described by the offset from the Derived-from to the solution frame. This offset is computed, if possible, when the solution is submitted. Computing this offset requires determining the position of the Derived-from frame in the Reference frame using the given View. Once this offset is computed, the reference View is no longer needed.

3.7.3. Derived-from and Orientations

The above discussion of the Derived-from frame is centered around position. Orientations also have a Derived-from frame, that should be used in the same way if the orientation is derived from image analysis.

However, most orientations are determined using an IMU or a sun-find. These methods of determining orientation provide what might be thought of as absolute orientations but are actually relative to the surface (north/down). For this reason, these solutions generally use the Site frame as their Derived-from. Orientations derived from looking at orbital imagery would use an orbital frame as their Derived-from.

3.7.4. Parent Views

Every view has a parent view, that is used to fill in the sparse RMC list of the child and make available a solution for every RMC. This is true until the root of the view tree is reached.

The root of the View tree is "telemetry". The "telemetry" view has no parent; it is expected to provide a value for every known RMC. It may in fact have a few holes (such as unsolicited moves) which are filled in by the child views, but sibling views not containing the unsolicited move will simply not see the missing RMC.

3.8. Are Sites Parallel?

The question as to whether site frames are parallel is a complicated one. On the one hand, sites are defined as +X pointing north and +Z pointing down according to the local gravity vector. On a curved surface such as Mars, this means site frames are not parallel; they mimic lines of longitude, which are not parallel to each other.

On the other hand, within the range of locations relevant to operations, the difference is negligible. The difference in Z due to planet curvature is on the order of 15cm over a 1km traverse, far less than the measurement error over that distance. Therefore, the rover onboard assumes site frames are parallel.

This is reasonable for operations; it is rare that data from over (or even close to) 1km away are used during operations. However, for precision large-scale mapping, it could make a difference.

PLACES remains agnostic on this issue, letting the user decide. The Site frame stores, like any other frame, both a position and an orientation relative to its reference frame. For the "telemetry" view, the orientation quaternion is set to identity (no rotation relative to the reference), which mimics the onboard software and makes all sites parallel. Operations users are free, however, to define a site solution that contains a rotation term, in order to compensate for planet curvature. If such solutions were defined in operations, they would show up in the PDS archive. In practice, this has never been done; Site frames are parallel in every view defined to date.

It should be noted that in practice, large-scale mapping to date has been done almost entirely using the Global frame. This renders the question of Site frame parallelism moot, because Site frames are not used. Planet curvature is accounted for by the map projection (Section 3.9.1).

3.9. Mapping of Orbital Images

Orbital images are referenced using a single index. Each orbital image stands on its own, defining a coordinate system relative to some parent (often the global frame, orbital(0), but possibly other orbital images).

Data associated with an orbital image include:

- Image identification (filename or URL)
- Pixel scale
- Size in pixels
- Location on the image (in pixel space) of the XYZ origin
- Standard descriptive metadata information (originator, description, etc.) as with any other solution ID or view.

These data, except for the descriptive metadata, are immutable once set.

Additionally, like any other frame, orbital images contain:

- Reference frame identification
- Transform from XYZ origin to the reference frame origin
- Derived-from frame (will be the same as reference frame)

The transform can include an offset and/or orientation just like any other frame reference. This allows multiple maps to all have North up yet have a rotation between them to cover large distances (see Section 3.8). This rotation will be identity (no rotation) in most cases, although rotation between Orbital frames is supported.

3.9.1. Orbital Image Map Projection

Orbital images must be in an equirectangular projection, with north up in the image. The line dimension of the image is aligned north/south with the sample dimension aligned east/west. There is a constant pixel scale (meters/pixel) in both directions, which is defined in the orbital image metadata. Mars 2020 uses a standard parallel of 18.4663, which goes through the Jezero Crater landing site, and minimizes distortion in the area of rover operations. The coordinate system origin is set to Airy-0 per IAU convention. Note particularly that northing=0 is at the equator, not the standard parallel.

While equirectangular projections are not generally considered good for mapping, rover operates in a relatively small area, and the use of the standard parallel means that shapes, areas, and distances are reasonably true. Angles between two points are true.

3.9.2. Projected Meters vs. True Meters

It is important to note that the east/west measurement, called "easting", is defined by meters *at the standard parallel*. Orbital images as used by the Mars 2020 localization science team, and thus by PLACES, use a rectangular projection, so that lines of longitude are parallel (as are lines of latitude). For convenience there are an equal number of meters between each line of longitude, regardless of latitude. This greatly simplifies calculations, and as a nice side-effect causes site frames to be parallel in the orbital projection.

These "easting" meters at a given latitude are related to true meters at the equator by the simple formula:

$$easting_meters = \frac{true_meters * \cos \phi_{ts}}{\cos \phi}$$

where ϕ is the planetocentric latitude and ϕ_{ts} is the standard parallel. The difference between easting meters and true meters is not significant in the area of rover operations, but is about 5.4 cm/meter at the equator.

Meters measured in the north-south direction ("northing") are true meters.

It should be noted that the rover works only with true meters, and does not take into account planet curvature. Thus the "telemetry" view (and only the "telemetry" view) is actually expressed in true meters and assumes a flat world. The localization error in telemetry, even with visual odometry turned on, is far greater than the true vs. easting meters difference, so in practice this distinction makes no difference.

3.9.3. Elevation, Z, Areoid, and Radial Distance

As noted in Section 2.1.4, Rover PLACES Z values are measured in a +Z-down coordinate system. They are thus the negative of cartographic elevations, which are measured +Z up. Throughout this document and the data, "Z" and "elevation" are used carefully to indicate the sign of the associated number. Note that for Mars 2020's entire area of operations, the Z value is positive, while elevations are negative (thus, the ground is below the reference surface). But this raises the

question, what is the reference surface for these elevations? It is an important question if one needs to convert to radius values (distance from the center of the planet).

Elevations in PLACES follow the conventions established by the Mars 2020 Localization Science team. They are referenced to the MOLA areoid (geoid in Earth terms). The areoid defines "sea level" on Mars, which is the shape of the surface of the water if Mars were filled to an average depth of 3396.190 km. It takes into account the local gravitational and rotational variations across the surface, and is technically the equipotential surface (gravitational plus rotational) whose average value at the equator is 3396.190 km. A full treatment of the MOLA areoid is beyond the scope of this document; see [4].

At the Mars 2020 landing site, the areoid is 3394507.076 m, which is about 1683 meters below the 3396190 m reference ellipsoid.

For convenience, the CSV data files report the areoid value, the elevation value, and the radius for each point. The radius is defined as the distance from the rover to the center of the planet, and is just the areoid plus the elevation.

3.10.Sources of Telemetry Data

The “telemetry” view will be filled in using as many data sources as possible. Any telemetry channel that provides position or orientation value for a given RMC is a potential source. The specific list of sources may expand over time but currently include:

- Image data (using the labels, derived from the Image Data Product Header (IDPH))
- Non-image data products (using the common telemetry product header)
- Kinematic state data products: general, and specific to arm, HGA, RSM, and mobility.
- EVRs

4. PDS Bundle Contents and Structure

The PDS delivery of PLACES data is contained in the mars2020_rover_places bundle. It consists of seven collections:

- data_localizations
- data_maps
- data_orbital
- data_mechanisms
- data_ancillary
- browse_maps
- document

Each collection is covered in a separate section below.

4.1. Sols Covered by a Delivery

PDS deliveries for Mars 2020 are typically defined by a sol range, for example 0-89, 90-119, 120-209, etc. However, PLACES works primarily in the RMC realm of sites, rather than sols. For that reason, any given PDS Rover PLACES distribution will contain all data for a range of Sites. This range of Sites is large enough to encompass the entire range of Sols. But it does mean that some additional data from outside the nominal sol range is necessarily provided as part of the delivery. Note that the last Site may not be complete if the rover operationally was still working in that Site when the archive release was generated. In that case, subsequent releases will add data to that Site.

4.2. Bundle Structure and File Types

PLACES data are delivered in the following structure, where each top-level directory is a collection:

```
<root>
| -- bundle.xml
| -- browse_maps
        | -- collection_browse_maps.xml
        | -- collection_browse_maps_inventory.csv
        | -- m20_orbital_dem.png
        | -- m20_orbital_dem.png.xml
        | -- m20_orbital_map.png
        | -- m20_orbital_map.png.xml
| -- data_ancillary
        | -- collection_data_ancillary.xml
        | -- collection_data_ancillary_inventory.csv
        | -- observations.csv
        | -- observations.csv.xml
        | -- sclk_table.csv
        | -- sclk_table.csv.xml
| -- data_localizations
```

```
        | -- collection_data_localizations.xml
        | -- collection_data_localizations_inventory.csv
        | -- view_metadata.csv
        | -- view_metadata.csv.xml
        | -- <view>.csv
        | -- <view>.csv.xml
        | -- <view>_raw
                | -- m20_<view>_roverframe_site_<n>.xmlx
                | -- m20_<view>_roverframe_site_<n>.xmlx.xml
                | -- m20_<view>_siteframe.xmlx
                | -- m20_<view>_siteframe.xmlx.xml
| -- data_maps
        | -- collection_data_maps.xml
        | -- collection_data_maps_inventory.csv
        | -- m20_orbital_dem.img
        | -- m20_orbital_dem.xml
        | -- m20_orbital_map.img
        | -- m20_orbital_map.xml
| -- data_mechanisms
        | -- collection_data_mechanisms.xml
        | -- collection_data_mechanisms_inventory.csv
        | -- mech_<mechanism>.csv
        | -- mech_<mechanism>.csv.xml
| -- data_orbital
        | -- collection_data_orbital.xml
        | -- collection_data_orbital_inventory.csv
        | -- orbital_origin.csv
        | -- orbital_origin.csv.xml
        | -- orbital_origin_global.csv
        | -- orbital_origin_global.csv.xml
        | -- orbital_<n>_metadata.csv
        | -- orbital_<n>_metadata.csv.xml
| -- document
        | -- collection_document.xml
        | -- collection_document_inventory.csv
        | -- <document_name>
        | -- <document_name>.xml
```

Values within `<angle_brackets>` are variable, and the associated files and directories are repeated several times with different values. See the specifics for each section.

The bundle label, collection label, and collection inventory are standard PDS4 files. See the PDS standards [5, 6, 7].

The files ending in `.xml` are PDS4 label files; there is one for every data file. The `.xml` file extension is appended to the data file filename, except in the case of images, where the `.img` extension is removed first. This convention matches that used for the Mars 2020 image bundles.

Data files are those ending in `.csv` (except for the collection inventories), `.img`, or `.xmlx`. Those ending in `.xmlx` are actually in XML format, but are renamed to avoid colliding with the PDS4 usage of `.xml` for label files. Data files are synonymous with data products for this bundle; all products consist of a single file and label.

Many of the files are in XML format. In order to present the data as originally entered, and reduce the possibility of translation error, the XML files are extractions directly out of the operational PLACES database. A few elements that are not relevant to the archive are removed, such as `href` tags, the `<?xml>` boilerplate, `xmlns` (namespace) attributes, and redundant sclk values (those in scientific notation). Otherwise, the original files are not edited. Note that there is no schema for these XML files as XML is just used as a data transport mechanism.

It should be noted that the PDS4 Processing Level of all data products in this bundle is Derived. See Table 1 for a description of processing levels.

Table 1. PDS Processing Level

PDS processing level	Description
Telemetry	An encoded byte stream used to transfer data from one or more instruments to temporary storage where the raw instrument data will be extracted. PDS does not archive telemetry data.
Raw	Original data from an instrument. If compression, reformatting, packetization, or other translation has been applied to facilitate data transmission or storage, those processes will be reversed so that the archived data are in a PDS approved archive format. Often called EDRs (Experimental Data Records).
Partially Processed	Data that have been processed beyond the raw stage but which have not yet reached calibrated status. These and more highly processed products are often called RDRs (Reduced Data Records).
Calibrated	Data converted to physical units, which makes values independent of the instrument.
Derived	Results that have been distilled from one or more calibrated data products (for example, maps, gravity or magnetic fields, or ring particle size distributions). Supplementary data, such as calibration tables or tables of viewing geometry, used to interpret observational data should also be classified as 'derived' data if not easily matched to one of the other three categories.

4.3. Versions

The Rover PLACES bundle consists mostly of files that accumulate data across the entire mission. In order to simplify the archiving process and reduce potential for errors related to incremental updates, the entire bundle is redelivered for each PDS release, including files that do not change between releases. Thus, each release is completely self-contained, with no need for any data from prior releases. Although prior releases are kept for historical purposes (this is an archive, after all), the latest available release should be used for all purposes.

To further simplify things, the version number of every file in the archive is the same and is tied to the PDS release number. For example, in PDS data release 3, all files will have version 3. (If adjustments need to be made post-release, the minor version could be used, which is generally 0). Note that because Rover PLACES data were not delivered until release 3, this means the versions start with 3.0.

Also, the filenames themselves are not versioned; the version is contained only in the label. The PDS Geosciences Node, which is responsible for hosting the Rover PLACES bundle, will manage the archival storage of older versions.

4.4. Data Formats and Ranges

Unless otherwise specified in the detailed description, the following data ranges apply to all data items of the given type.

`X, Y, Z values`: Double-precision floating point. Data range is unlimited. Measured in meters. A flag value of -9999 indicates the value is not available.

`Northing, easting, elevation values`: same as XYZ.

`Quaternions`: Quaternions represent a rotation in 3-D space, in the order (s, v1, v2, v3), where:

$$s = \cos\left(\frac{\theta}{2}\right)$$

$$v_n = \sin\left(\frac{\theta}{2}\right) * a_n$$

and θ is the angle of rotation and a is a unit vector defining the axis of rotation. All quaternions are unit quaternions, thus the data range is -1.0 ... +1.0. A flag value of -9999 indicates the value is not available.

`RMC components`: Integers starting at 0 with no specific upper bound, with -1 indicating no value.

`Angles`: Represented as floating-point numbers in degrees, typically -360.0 to +360.0. Some angles have different valid ranges, including > 360. A flag value of -9999 indicates the value is not available.

`Latitude`: Double-precision floating-point, measured in degrees from -90.0 to +90.0.

`Longitude`: Double-precision floating-point, measured in degrees from -360.0 to +360.0. Standard east-positive longitude is used.

`Pixel values`: Floating-point values with no specific range (pixel coordinates may be off a given image, including negative). Note that the center of the upper-left pixel of an image has coordinates line=1.0 and sample=1.0 for 1-based coordinates (the default), and line=0.0, sample=0.0 for 0-based coordinates (when specified).

5. Data File (Collection) Descriptions

This section describes in detail the data files in the Rover PLACES delivery. It is organized by collection (top-level directory).

5.1. Collection: data_localization

A large set of data is provided for each View that is extracted for PDS. The format and layout of these data are standardized; i.e. they are the same for each View.

5.1.1. Included Views

The set of views that is included may change from delivery to delivery. However, the current set is described below, and consists of three views. Additional views will be added to support solutions derived from auto-localization in the future (as distinct from the current manual localization process).

5.1.1.1. telemetry

The "telemetry" view contains locations and orientations as reported by the rover. They are not corrected on the ground and thus represent the raw data from which other localizations are derived. The effects of wheel slippage and other cumulative errors mean that, in a broad sense, the telemetry positions do not match any map or other reference. The telemetry view should thus not generally be used directly; one of the other views is almost always better. However, this view is of considerable historical interest because it preserves the state of rover knowledge before ground adjustment. Also, in a local sense, such as within one sol's worth of driving, the telemetry view does a reasonable job of showing the relative traverse for the day. Finally, comparing telemetry to a localized view can provide some idea of how much the rover slipped.

The rover is capable of updating its pose knowledge, for example by using visual odometry, reading the IMU, etc. These updates are tracked using the Pose counter in the RMC (see section 2.2.3). The pose counter acts like a solution ID in that different pose values represent different states of pose knowledge by the rover. A complete treatment of how Pose should be handled is beyond the scope of this document. Nevertheless, a useful rule of thumb is to use the highest numbered Pose for any given Site/Drive. This reflects the results of visodom and other updates. Unfortunately, the reason for any given Pose update is not available to, nor tracked by, PLACES.

5.1.1.2. best_tactical

The "best_tactical" view contains locations as updated by the Mars 2020 Localization team. This team, headed by the Mars 2020 Localization Scientist, determines updated positions for the rover after every drive by comparing orthorectified images from the rover navcam with satellite (HiRISE) photos. These locations are specifically designed to match the map and DEM cutouts (Section 5.2) included with this delivery. The localized positions have an expected accuracy of 0.5m compared to the base map.

The best_tactical view is sparse, containing just those locations where localization was performed (typically the last drive of a Sol, although localization is occasionally done more than once on a Sol). It does not contain intermediate positions during a drive.

Z values (negative of elevation) for each location are derived from the DEM for the base map, created from orbital HiRISE data (see Section 5.2). The Z value is simply the negative of the DEM value at the given XY coordinate (because the +Z axis is down). The telemetered Z value (relative to the Site origin) is ignored for this purpose.

The localization team does not currently adjust orientation. Thus, the orientations included in the solution are copied straight from the telemetry view.

5.1.1.3. best_interp

The "best_interp" view contains all the localized solutions that are in best_tactical. In addition, it includes "interpolated" values for all drive segments in between those locations. Thus, every site/drive location seen in telemetry, should have an entry in best_interp, creating a continuous drive path.

Interpolation proceeds as follows. Given two endpoints (localized positions from best_tactical), it gathers all telemetry entries (highest pose only) between those points. The start is subtracted from the end in both the best_tactical and telemetry views to get vectors. The vectors are then compared to determine an angle of rotation between them and a scale factor. These values are then used to transform each telemetry point:

$$\vec{V}_t = \vec{F}_t - \vec{O}_t$$

$$\hat{V}_t = \frac{\vec{V}_t}{\|\vec{V}_t\|}$$

$$\vec{V}_b = \vec{F}_b - \vec{O}_b$$

$$\hat{V}_b = \frac{\vec{V}_b}{\|\vec{V}_b\|}$$

$$\theta = \cos^{-1}\left(\hat{V}_t \cdot \hat{V}_b\right)$$

$$\hat{A} = \hat{V}_t \times \hat{V}_b$$

$$Q = \left(\cos\left(\frac{\theta}{2}\right), \sin\left(\frac{\theta}{2}\right)\hat{A}\right)$$

$$s = \max \frac{\left(\vec{V}_b\right)_{x,y,z}}{\left(Q * \vec{V}_t\right)_{x,y,z}}$$

$$\vec{P}_i = s\left(Q * \left(\vec{P}_t - \vec{O}_t\right)\right) + \vec{O}_b$$

where

$\vec{O}_t, \vec{F}_t$: original and final points, in telemetry
$\vec{O}_b, \vec{F}_b$: original and final points, in best_tactical
$\vec{V}_t, \vec{V}_b$: vector from original to final points
$\hat{V}_t, \hat{V}_b$: unit vector from original to final points
θ : angle by which path is rotated from telemetry to best_tactical
$\hat{A}$: axis of rotation
Q : quaternion expressing rotation from $\hat{V}_t$ to $\hat{V}_b$
s : scale factor for path
$\vec{P}_t$: Point to convert, in telemetry
$\vec{P}_i$: Final converted point, for best_interp

This method preserves a continuous path based on the shape of the path in telemetry. It implicitly assumes all errors are evenly distributed along the path. This is of course not true; therefore, the interpolation is an approximation. However, the true distribution of errors is not known, so no exact answer is possible.

Note that Z values (negative of elevations) are interpolated along with the X and Y positions. This means the best_interp Z value will follow the curve defined by telemetry for Z, while meeting the DEM at the endpoints. These are higher resolution data than is available from the orbital DEM and should therefore track the surface better. However, the endpoints are constrained to DEM elevations, and this may introduce some distortions in the Z shape at the endpoints.

As with best_tactical, orientations are not currently adjusted as part of this process and are copied directly from telemetry.

5.1.2. View Metadata

Metadata for all views, as entered by the user who created it, are contained in a single file, "`view_metadata.xml`". Each view gets one line in this file. An example is shown below.

```
view,group,user,add_date,creation_date,description
best_interp,tactical,kgrimes,2021-02-18,2021-02-18,Populated by interp script
best_tactical,tactical,kgrimes,2021-02-18,2021-02-18,Solutions localized by mapping
specialist
telemetry,telemetry,kgrimes,2021-02-18,2021-02-18,Telemetry
```

Field descriptions:

`view`: Contains the view name (id).

`group`: Contains the group the view is in. Groups are a database feature used to assign privileges and are not particularly important for the archive.

`user`: contains the user id of the person who created the view.

`add_date`: contains the date the view was added to the database.

`creation_date`: Contains the date the view was created. Essentially the same as `add_date`.

`description`: Contains a short description of the view. Section 5.1.1 has a much more complete description.

5.1.3. Per-View CSV File

The CSV, or Comma-Separated-Value, file will likely be the most useful file for most users. This file is named "***view***`.csv`".

The file is a standard text file. There are 28 fields per entry (line), separated by commas. The first line contains column headings; all other lines contain data. The columns are described below. But first, an example, from the best_interp view:

```
frame,site,drive,pose,landing_x,landing_y,landing_z,northing,easting,planetocentric
_latitude,planetodetic_latitude,longitude,elevation,map_pixel_line,map_pixel_sample
,dem_pixel_line,dem_pixel_sample,roll,pitch,yaw,quat_s,quat_v1,quat_v2,quat_v3,areo
id,radius,sclk,sol

SITE,1,-1,-1,0.000,0.000,0.000,1093299.695,4354494.086,18.444627146,18.346169209,
77.450885729,-2569.910,2161.22,4296.34,540.30,1074.09,-9999.000,-9999.000,
-9999.000,-9999.000000000,-9999.000000000,-9999.000000000,-9999.000000000,
3394507.076,3391937.166,0,-1

ROVER,1,0,0,0.000,0.000,0.000,1093299.695,4354494.086,18.444627146,18.346169209,
77.450885729,-2569.910,2161.22,4296.34,540.30,1074.09,-9999.000,-9999.000,
-9999.000,-9999.000000000,-9999.000000000,-9999.000000000,-9999.000000000,
3394507.076,3391937.166,666952977,0

...

SITE,3,-1,-
1,0.000,0.000,0.000,1093299.695,4354494.086,18.444627146,18.346169209,77.450885729,
-2569.910,2161.22,4296.34,540.30,1074.09,-9999.000,-9999.000,-9999.000,
-9999.000000000,-9999.000000000,-9999.000000000,-9999.000000000,3394507.076,
3391937.166,667754529,9

ROVER,3,0,0,0.000,0.000,0.000,1093299.695,4354494.086,18.444627146,18.346169209,
77.450885729,-2569.910,2161.22,4296.34,540.30,1074.09,-1.197,-0.030,132.780,
0.400486900,-0.003942070,-0.009678784,0.916242900,3394507.076,3391937.166,
667751508,9

...

ROVER,3,44,2,-1.250,1.336,-0.004,1093298.445,4354495.422,18.444606063,
18.346169209,77.450909495,-2569.906,2166.22,4301.69,541.55,1075.42,-1.715,0.269,
132.834,0.400002100,-0.008141161,-0.012771733,0.916389050,3394507.081,3391937.175,
668187227,14
```

Each field means the following:

`frame`: Frame type, SITE or ROVER.

`site`: Site number from the RMC.

`drive`: Drive number from the RMC. Will be -1 for SITE entries.

`pose`: Pose number from the RMC. Will be -1 for SITE entries, and for any view other than telemetry. Exception: for best_tactical and best_interp, an extra entry is added for ROVER(*n*,0) to show the rover orientation at the location where a Site frame was declared, as a convenience (this entry does not exist in the database, because the Site is localized, not Drive=0). This entry will have the highest pose number available for that Site *n* Drive 0, which is the one used for the entry.

`landing_x`: X coordinate relative to the landing site (SITE 1). +X points north.

`landing_y`: Y coordinate relative to the landing site (SITE 1). +Y points east.

`landing_z`: Z coordinate relative to the landing site (SITE 1). +Z points ***down***.

`northing`: Meters north of the equator (or south, if negative). Same as the X coordinate, in PLACES parlance. Most cartographers would call this +Y.

`easting`: Meters east of the prime meridian, as measured at the projection's first standard parallel (see Section 3.9.1). Same as the Y coordinate in PLACES parlance. Most cartographers would call this +X.

`planetocentric_latitude`: Latitude of the point (in degrees), measured using a planetocentric system. Planetocentric coordinates are measured as angles from the center of the planet. Latitude is computed from northing (x) using the formula:

$$\phi_{pc} = \frac{x}{R_e} \cdot \frac{180}{\pi}$$

where R_e is the ellipsoid radius, or 3396190 meters.

`planetodetic_latitude`: Latitude of the point (in degrees), measured using a planetodetic system. Planetodetic coordinates are measured as the tangent to the ellipsoidal surface at that point. Planetodetic latitude is computed using the formula:

$$\phi_{pd} = \tan^{-1}\left(\left(\tan \phi_{pc}\right) \left(\frac{R_e}{R_s}\right)^2 \right)$$

where ϕ_{pc} is the planetocentric latitude, R_e is the radius of the ellipsoid (3396190), and R_s is the radius of the sphere (3376200).

`longitude`: Longitude of the point (in degrees). Longitude is computed from easting (y) using the formula:

$$\theta = \frac{y}{R_e \cos\left(\phi_1 \left({}^{\pi}/_{180}\right)\right)} \cdot \frac{180}{\pi}$$

where R_e is the ellipsoid radius, or 3396190 meters and and ϕ_1 is the standard parallel, 18.4663 degrees. Note that there is no difference in longitude between the planetocentric and planetodetic systems, because Mars is modeled as a spheroid where the equator is a circle, not a triaxial ellipsoid.

`elevation`: Elevation of the point relative to the datum (areoid). This uses the cartographic definition of elevation, so positive elevation is ***up***. It is thus computed as the negative of the global-frame Z value.

`map_pixel_line`: 0-based line number of the point in the base map cutout image (see Section 5.2). Computed from northing (global-frame x) using the formula:

$$l_{map} = -\left(\frac{x - min_x}{x_scale}\right)$$

where *min_x* and *x_scale* come from the file `data_maps/m20_orbital_map.xml` in the `x_axis_minimum` and `pixel_resolution_x` keywords, respectively.

`map_pixel_sample`: 0-based sample number of the point in the base map cutout image (see Section 5.2). Computed from easting (global-frame y) using the formula:

$$s_{map} = -\left(\frac{y - min_y}{y_scale}\right)$$

where *min_y* and *y_scale* come from the file `data_maps/m20_orbital_map.xml` in the `y_axis_minimum` and `pixel_resolution_y` keywords, respectively.

`dem_pixel_line`: 0-based line number of the point in the DEM cutout image (see Section 5.2). Computed the same as `map_pixel_line` except the file `m20_orbital_dem.xml` is used as the source of the values.

`dem_pixel_sample`: 0-based sample number of the point in the DEM cutout image (see Section 5.2). Computed the same as `map_pixel_sample` except the file `m20_orbital_dem.xml` is used as the source of the values.

`roll`: Specifies the roll value (in degrees) of the rover at this location. Computed by decomposing the orientation quaternion into Euler angles. Roll is a clockwise rotation around the Rover frame +X axis (positive means left side is up). Data range -180 to +180 degrees. Value is -9999 if unknown.

`pitch`: Specifies the pitch value (in degrees) of the rover at this location. Computed by decomposing the orientation quaternion into Euler angles. Pitch is a clockwise rotation around

the Rover frame +Y axis (positive means nose is up). Data range -90 to +90 degrees. Value is -9999 if unknown.

`yaw`: Specifies yaw value (or heading, in degrees) of the rover at this location. Computed by decomposing the orientation quaternion into Euler angles. Yaw is a clockwise rotation about the Rover frame +Z axis. Because +Z points down, yaw is equivalent to heading, with north at 0 degrees, east at 90, south at 180, and west at 270. Data range from 0 to +360 degrees. Value is -9999 if unknown.

`quat_s`: Scalar component of rover orientation quaternion. See Section 4.4 for quaternion definition. Value is -9999 if unknown.

`quat_v1`: Vector-1 component of rover orientation quaternion. See Section 4.4 for quaternion definition. Value is -9999 if unknown.

`quat_v2`: Vector-2 component of rover orientation quaternion. See Section 4.4 for quaternion definition. Value is -9999 if unknown.

`quat_v3`: Vector-3 component of rover orientation quaternion. See Section 4.4 for quaternion definition. Value is -9999 if unknown.

`areoid`: Value of the areoid at this location, in meters. See Section 3.9.3.

`radius`: Value of the radius at this location (distance from rover to center of planet) in meters. See section 3.9.3

`sclk`: Earliest spacecraft clock (sclk) seen for the site/drive on this line. This can be derived via a lookup in the sclk table (Section 5.6.2) but is repeated here for convenience. Pose is ignored when doing this lookup. Fractional values are truncated down to integer.

`sol`: Solar day (sol) number associated with the sclk. This value is derived from the sclk using the SPICE "chronos" program.

Note that each `.csv` file has an associated `.xml` file, which describes the format of the CSV file according to PDS standards. The `.xml` file is not further described here.

5.1.4. Raw Data For Each View

Each view has a directory, named "***view***`_raw`", which contains the data entered into PLACES for the view, in the manner in which it was entered. This means there is no consistency in terms of reference frames. If one localization was done relative to a global frame while the next was done relative to a site frame, those raw values are reported.

The usefulness of the raw data is that it reflects what the user entered, before any processing. Thus, any bugs that might occur during processing (e.g. to translate to the global frame, below) do not impact this version. Also, all the known relationships are preserved, so if the PDS user wanted to, for example. adjust the location of a site, the effects of that on downstream solutions (derived

from that) could be re-computed. However, the potential inconsistencies in representation make this directory somewhat less useful for the casual user.

Note that for the telemetry view, there are no inconsistencies in representation, because the rover always reports data in the same way. All reference frames are the enclosing Site, or the previous Site for site frames themselves.

Within the directory, there are two types of data files (each with their own .xml label of course):

```
m20_view_siteframe.xmlx
m20_view_roverframe_site_*.xmlx
```

There is one `siteframe` file, containing all the Site frame definitions. Then for each Site, there is a `roverframe` file for that site, containing all the positions in the site. Both files share the same format.

These files are in XML format, but are named `.xmlx` in order to not conflict with the PDS4 usage of `.xml` to represent labels.

The site and rover frame files consist of a list of `<solution>` entries. An example is presented here:

```
<views>
   <paging offset="0" page-size="-1" total-entries="873"/>
   <solution>
      <rmc actual_sclk="6.7849014E8" drive="2882" frame="ROVER" pose="10"
           sclk_printable="678490140" site="4"/>
      <offset x="-509.0047" y="-27.621834" z="1.6447364">
         <solution_metadata>
            <derived_from actual_sclk="6.78485725E8" date_added="2021-07-02 12:28:47.0"
                 drive="2878" frame="ROVER" pose="4" site="4"/>
            <reference_rmc date_added="2021-05-16 04:04:42.0" frame="SITE" site="4"/>
            <reference_view>telemetry</reference_view>
         </solution_metadata>
      </offset>
      <orientation s="0.07322795" v1="-0.0034846405" v2="0.00942334" v3="-0.9972646">
         <solution_metadata>
            <derived_from date_added="2021-05-16 04:04:42.0" frame="SITE" site="4"/>
            <reference_rmc date_added="2021-05-16 04:04:42.0" frame="SITE" site="4"/>
            <reference_view>telemetry</reference_view>
         </solution_metadata>
         <error s="-1.0" v1="-1.0" v2="-1.0" v3="-1.0"/>
      </orientation>
   </solution>
   <solution>
      ...
   </solution>
   ...
</views>
```

`<paging>`: The database can break responses into pages, but this capability is not used for the PLACES archive. This entry can be ignored, although the total-entries could be useful.

`<solution>`: There is a `solution` element for each unique RMC in the view.

<rmc>: Defines the RMC and frame for this solution. The sclk_printable is just an easier to use form of actual_sclk without the scientific notation – which in turn is simply the sclk from the data product from which the information was derived when known (that information is not always available).

<offset>: Provides the XYZ coordinates (position) of this solution (relative to the reference frame).

<orientation>: Provides the quaternion (s is the scalar component; v1, v2, and v3 are the vector components) specifying the orientation for this solution (relative to the reference frame). Mathematically the quaternion should be interpreted as follows: Given a vector expressed in the current frame, multiplication by this quaternion will give the same vector as expressed in the reference frame.

<solution_metadata>: Defines the reference and derived-from frames for this solution. These can be different for offset and orientation.

<derived_from>: Defines the derived-from frame (see Section 3.7) for this solution.

<reference_rmc>: Defines the reference frame (see Section2.1.5) for this solution.

<reference_view>: Defines the reference view for this solution.

5.2. Colletion: data_maps

The data_maps collection contains a cutout of the "base map" that covers the area traversed by the rover during the time represented by this delivery. This base map is the reference used for all localizations by the M20 Project. It is important to realize that the localized positions, including latitude/longitude in the CSV file, are created using this map. If the latitude/longitude are transferred to another map, the registration will only be as good as the coregistration of the two maps. That is the primary reason the map is being included: to allow users to unambiguously identify localized positions by features, and decide for themselves how good the coregistration is between the M20 base map and the map they want to use.

The source base map is a 52,000 line by 36,000 sample mosaic at 0.25 m/pixel resolution covering the landing site in Jezero Crater, the Jezero Delta, and surrounding areas. It covers the entire expected M20 operational area at Jezero. It was made by the M20 Localization Team using mostly MRO HiRISE orthophoto data generated by the USGS, with holes filled by MRO CTX low-nadir angle map projected images, and registered to the Mars Express HRSC 50m/pixel DEM which is in turn registered to the MOLA gridded DEM.

Full metadata for the map, including source images and pointers to the full image and DEM, is available here:

https://astrogeology.usgs.gov/search/map/Mars/Mars2020/JEZ_hirise_soc_006_orthoMosaic_25cm_Eqc_latTs0_lon0_first

The cutout is determined by looking at the minimum and maximum extents of the "best_tactical" view for the duration covered by this release, plus ~500 meters on all sides.

Additionally, the collection contains a similar cutout of the Digital Elevation Model (DEM) map. This DEM contains the elevation (in meters) for every point in the map. The DEM is 13,000 lines by 9,000 samples at 1.0 m/pixel resolution covering the same area as the base map. It was also made by the M20 Localization Team using USGS MRO HiRISE DEMs, backfilled with a single CTX DEM created at JPL using Socet Set, and is similarly coregistered using the associated orthophotos.

There are four files in the `DATA/MAPS` directory, described below.

5.2.1. m20_orbital_map.img, m20_orbital_map.xml

The `m20_orbital_map.img` file contains the actual map cutout. It is a standard byte-format dual-label VICAR/ODL image as described in the camera SIS [1]. The `m20_orbital_map.xml` file is the PDS4 label for the image. The PDS4 label is the definitive label, but it is derived from the VICAR/ODL label, so the semantic content is the same. Thus, either label can be used.

5.2.2. m20_orbital_dem.img, m20_orbital_map.xml

These files contain the DEM cutout. They are identical in format to the `m20_orbital_map.*` files, except that the image is in 32-bit floating point format.

5.2.3. ODL Map Labels

An example of the ODL map label (attached to the image) is shown below.

```
ODL_VERSION_ID                    = ODL3

/* FILE DATA ELEMENTS */

RECORD_TYPE                       = FIXED_LENGTH
RECORD_BYTES                      = 6680
FILE_RECORDS                      = 8322
LABEL_RECORDS                     = 1

/* POINTERS TO DATA OBJECTS */

^IMAGE_HEADER                = 2
^IMAGE                       = 3
OBJECT                            = IMAGE
  INTERCHANGE_FORMAT              = BINARY
  LINES                           = 8320
  LINE_SAMPLES                    = 6680
  SAMPLE_TYPE                     = UNSIGNED_INTEGER
  SAMPLE_BITS                     = 8
  BANDS                           = 1
  BAND_STORAGE_TYPE               = BAND_SEQUENTIAL
END_OBJECT                        = IMAGE

/* IDENTIFICATION DATA ELEMENTS */

 INSTRUMENT_HOST_NAME             = "MARS 2020"
 MISSION_NAME                     = "MARS 2020"
 INSTRUMENT_NAME                  = "N/A"
 PRODUCT_CREATION_TIME            = 2021-12-21T22:52:15
 PRODUCT_ID                       = "m20_orbital_map"
```

```
 PRODUCT_VERSION_ID               = "1.0"
 TARGET_NAME                      = "MARS"
 TARGET_TYPE                      = PLANET
 MISSION_PHASE_NAME               = "PRIMARY SURFACE MISSION"
 SPACECRAFT_CLOCK_START_COUNT     = "666952977.663"
 SPACECRAFT_CLOCK_STOP_COUNT      = "688959835"
 START_TIME                       = 2021-02-18T20:44:27
 STOP_TIME                        = 2021-10-31T13:48:19
 DESCRIPTION                      = "Data from base map derived from MRO
                                     HiRISE and CTX.  See SIS for details"
 PDS4_BUNDLE                      = "mars2020_rover_places"
 PDS4_COLLECTION                  = "data_maps"
 STANDARD_MISSION_FILENAME        = FALSE
 VERSION_NUMBER                   = 3
 GEOMETRY_STATE                   = NONE

/* SURFACE PROJECTION DATA ELEMENTS */

GROUP                             = SURFACE_PROJECTION_PARMS
  MAP_PROJECTION_TYPE             = ORTHORECTIFIED
  MAP_SCALE                       = (0.25 <m/pixel>,0.25 <m/pixel>)
  REFERENCE_COORD_SYSTEM_NAME     = "ORBITAL"
  REFERENCE_COORD_SYS_INDEX_NAME  = "INDEX"
  REFERENCE_COORD_SYSTEM_INDEX    = 0
  X_AXIS_MINIMUM                  = 1091760.0
  X_AXIS_MAXIMUM                  = 1093840.0
  Y_AXIS_MINIMUM                  = 4353420.0
  Y_AXIS_MAXIMUM                  = 4355090.0
  CENTER_LATITUDE                 = 18.436196989
  CENTER_LONGITUDE                = 77.446633244
END_GROUP                         = SURFACE_PROJECTION_PARMS

/* IMAGE MAP PROJECTION DATA ELEMENTS */

OBJECT                            = IMAGE_MAP_PROJECTION
  A_AXIS_RADIUS                   = 3396.19 <km>
  COORDINATE_SYSTEM_NAME          = "PLANETOCENTRIC"
  MAP_PROJECTION_TYPE             = EQUIRECTANGULAR
  LINE_PROJECTION_OFFSET          = 4375359.5
  SAMPLE_PROJECTION_OFFSET        = -17413680.5
  MAP_SCALE                       = 0.25 <m/pixel>
  LATITUDE_ORIGIN                 = 0.0
  ^DATA_SET_MAP_PROJECTION        = "DSMAP.CAT"
  B_AXIS_RADIUS                   = 3396.19 <km>
  C_AXIS_RADIUS                   = 3396.19 <km>
  CENTER_LATITUDE                 = 18.4663
  CENTER_LONGITUDE                = 0.0
  COORDINATE_SYSTEM_TYPE          = "BODY-FIXED ROTATING"
  LINE_FIRST_PIXEL                = 1
  LINE_LAST_PIXEL                 = 8320
  MAP_PROJECTION_ROTATION         = 0.0
  MAP_RESOLUTION                  = 237098.7900932249 <pixel/deg>
  POSITIVE_LONGITUDE_DIRECTION    = EAST
  SAMPLE_FIRST_PIXEL              = 1
  SAMPLE_LAST_PIXEL               = 6680
  MINIMUM_LATITUDE                = 18.41865155989587
  MAXIMUM_LATITUDE                = 18.45374241800075
  WESTERNMOST_LONGITUDE           = 77.431781578217
  EASTERNMOST_LONGITUDE           = 77.46148490921553
END_OBJECT                        = IMAGE_MAP_PROJECTION

/* IMAGE HEADER DATA ELEMENTS */
```

```
OBJECT                          = IMAGE_HEADER
 HEADER_TYPE                    = VICAR2
 INTERCHANGE_FORMAT             = ASCII
 BYTES                          = 6680
 ^DESCRIPTION                   = "VICAR2.TXT"
END_OBJECT                      = IMAGE_HEADER
END
```

Note that the actual map extents and image sizes will differ from the example above, depending on the size of the cutout needed for the specific PLACES delivery. Also note that the label refers to M20 rather than MRO even though the map is originally derived from MRO data. This is because the images have been vertically and horizontally georeferenced by the M20 Localization Team to become the base map, and the cutout provided here is specifically sized to match the extent of the M20 data contained in PLACES.

The image is compliant with the camera SIS with the following exceptions:

- The filename is constant and does not follow the camera SIS conventions.
- The label is minimal, containing only map projection information and the minimum descriptive information needed.
- The x/y axis minimum and maximum values specify the corner coordinates in global northing (x) and easting (y) frame and refer to the outside edges of the respective pixels. See the formulas in Section 5.1.3 for converting these to latitude/longitude.
- REFERENCE_COORD_SYSTEM_NAME is ORBITAL (index of 0), which is the global northing (x)/easting (y) frame. See Section 3.9.1.
- CENTER_LATITUDE (planetocentric) and CENTER_LONGITUDE have been added for convenience. However, the corner points should be considered the definitive location reference. Note that these keywords mean something very different in the IMAGE_MAP_PROJECTION group, below.
- There are several keywords in IDENTIFICATION (starting with PDS4_BUNDLE) that are used solely to control the PDS4 label creation software.
- The IMAGE_MAP_PROJECTION object has been added to facilitate using the map images in tools such as GDAL (http://gdal.org). The additional keywords in this object (which are standard PDS 3 keywords) are listed and briefly described below in the context of this product only.
    - MAP_PROJECTION_TYPE: declares the projection as EQUIRECTANGULAR.
    - MAP_SCALE: the pixel size; pixels are square in this case.
    - A_AXIS_RADIUS, B_AXIS_RADIUS, C_AXIS_RADIUS: sets the Mars spherical radius to 3393190 m in the datum; all three are the same.

- COORDINATE_SYSTEM_NAME: sets the latitude type to PLANETOCENTRIC.
- LINE_PROJECTION_OFFSET: the offset in the line direction from the projection origin (latitude=0.0) to the image upper left corner in pixels (i.e. the image coordinate of latitude=0.0). Values are positive to the right and down (thus they are inverted from what one might expect, because northing is positive up).
- SAMPLE_PROJECTION_OFFSET: the offset in the sample direction from the projection origin (longitude=0.0) to the image upper left corner in pixels (i.e. the image coordinate of longitude=0.0). Values are positive to the right and down.
- ^DATA_SET_MAP_PROJECTION: obsolete.
- CENTER_LATITUDE: Defines the standard parallel for the equirectangular projection (see Section 3.9.1).
- CENTER_LONGITUDE: Defines the origin for the equirectangular projection. Always 0 in these products.
- COORDINATE_SYSTEM_TYPE: Constant "BODY-FIXED ROTATING".
- LINE_FIRST_PIXEL, SAMPLE_FIRST_PIXEL: Always 1,1.
- LINE_LAST_PIXEL, SAMPLE_LAST_IPXEL: Same as LINES and LINE_SAMPLES in the IMAGE object.
- MAP_PROJECTION_ROTATION: Always 0.0.
- MAP_RESOLUTION: Resolution of map in pixels/degree. Computed as:
$$\frac{2\pi\left(\text{A_AXIS_RADIUS}\times 1000\right)}{360\times \text{MAP_SCALE}}$$
- POSITIVE_LONGITUDE_DIRECTION: Constant "EAST".
- MINIMUM_LATITUDE, MAXIMUM_LATITUDE: Computed by converting X_AXIS_MINIMUM and X_AXIS_MAXIMUM to planetocentric latitude (see Section 5.1.3).
- EASTERNMOST_LONGITUDE, WESTERNMOST_LONGITUDE: Computed by converting Y_AXIS_MINIMUM and Y_AXIS_MAXIMUM to longitude (see Section 5.1.3).

5.2.4. PDS4 Map Labels

The PDS4 map labels are fully compliant with the PDS standard. As such, they are not discussed in detail here, but see Appendix A for label descriptions.

One unusual thing to note: there are two <cart:Cartography> sections in the label. These two sections describe the projection of the image in two different ways (they are compatible, with the same pixels).

The first section defines the projection as an orbital map, using the Equirectangular projection. This is derived from the IMAGE_MAP_PROJECTION group defined above. This can be used with standard mapping software.

The second section defines the projection as a surface (in-situ) map, similar to Orthorectified rover mosaics. This can be used with software that processes rover in-situ orthorectified mosaics.

5.2.5. Using localization data in GIS programs

The orbital datasets in the data_maps collection can be used directly in most GIS programs (e.g. ESRI ArcGIS) via either the VICAR, ODL, or PDS4 labels. All the pertinent image and projection attributes are retained in the labels, making it natively readable by most mapping software.

Below is an example using the GDAL program gdal_translate (version 2.5) to convert the default PDS format to a GeoTIFF with the correct extent:

```
gdal_translate m20_orbital_map.img m20_orbital_map.tif

gdal_translate m20_orbital_dem.img m20_orbital_dem.tif
```

The GeoTIFFs above have the correct extent in ArcGIS.

5.3. Collection: browse_maps

The browse_maps collection contains browse versions of the maps in the data_maps collection, in png format. There are only two images, with associated labels.

The DEM has been converted from float to byte for the browse collection, using a min-max stretch. As such, it is not usable to determine absolute elevations, but could be used to color-code differences in elevation using a pseudocolor display.

5.4. Collection: data_orbital

The data_orbital directory contains information on all the ORBITAL frames defined in PLACES. Orbital frames generally correspond to a map image but can also be defined for other reasons. For example, orbital(1) is the base map (see Section 5.2.1) while orbital(2) is the DEM that goes along with it (it's a different "frame" because the resolutions differ). Orbital(3) is a mosaic made using the Rover Descent Camera images (the mosaic itself is not delivered as part of Rover PLACES, but the frame defining it is included here). Note that the index number (other than 0, which is the global frame) is completely arbitrary and is whatever was assigned during operations. They have no relationship to rover Site frames.

Unlike rover locations, the location of an orbital map frame is known when it is created. Therefore, they do not generally have multiple solutions. Although multiple solutions are allowed by the database, they have not been used and are not reflected in the orbital data for PDS.

There is one file for each orbital frame, and one summary file, all described below.

5.4.1. Metadata

The `orbital_`***`n`***`_metadata.csv` files contain keyword-value pairs of metadata that were entered when each orbital frame was created. There is no formal standardization of the keywords (any keyword/value pair can be added by the user during operations), but the common ones are described below.

These files are keyword-value pairs rather than PDS4 labels because a) the lack of standardization would make converting to labels challenging, and b) there is no specific product to be labeled, because the orbital frames are abstract concepts. The orbital map cutouts are just that – cutouts, not the entire image described by the orbital frame definition.

An example of this file (for orbital(1)) follows:

```
key,value
pixel_type,"unsigned_integer"
center_longitude,"0"
coord_sys_definition,"+X is North, +Y is East, +Z is Down"
x_scale,"0.25"
description,"25cm M20 basemap Visible"
lower_right_easting_m,"4358000"
pixel_depth,"8"
latitude_type,"planetocentric"
lower_right_northing_m,"1090000"
z_scale,"1.0"
pixel_definitions,"Line/samp of (0,0) is the upper-left corner of the upper-left
 pixel"
false_northing,"0"
false_easting,"0"
projection,"Equirectangular"
lines,"52000"
projection_units,"meters"
standard_parallel,"18.4663"
upper_left_easting_m,"4345000"
origin_sample,"0"
samples,"36000"
center_latitude,"0"
ellipsoid_radius,"3396190"
filename,"M20_PrimeMission_HiRISE_CLR_25cm.tif"
origin_line,"0"
y_scale,"0.25"
upper_left_northing_m,"1099000"
filename_clr,"M20_PrimeMission_HiRISE_CLR_25cm.tif"
filename_orr,"M20_PrimeMission_HiRISE_ORR_25cm.tif"
```

Some of the common keywords (in column 1) are described below. As mentioned above, the set of keywords is not standardized so there may be others not described here, and not all these will be present in all frames.

`center_latitude`: Latitude of the center of the projection. Always 0 for the global frame.

`center_longitude`: Longitude of the center of the projection. Always 0 for the global frame.

`coord_sys_definition`: A text description of the coordinate system orientation.

`description`: A text description of the frame.

`ellipsoid_radius`: Radius of the Mars ellipsoid (in meters) used for latitude/longitude conversion.

`filename`: Filename of the (primary) image described by this frame. This file is not necessarily available in PDS.

`formula`: Describes how to use the values.

`latitude_type`: Planetocentric or planetodetic.

`lines`: Number of lines in the image.

`lower_right_easting_m`: Easting (in meters, see Section 3.9.1) of the lower right corner.

`lower_right_northing_m`: Northing of the lower right corner.

`origin_line`: Line number in the image of the origin (i.e. the point defined by the `orbital_origin.csv` file). Lines start counting at 0 in this context (see pixel_definitions).

`origin_sample`: Sample number in the image of the origin (i.e. the point defined by the `orbital_origin.csv` file). Samples start counting at 0 in this context (see pixel_definitions).

`pixel_definitions`: Describes how pixels are defined.

`pixel_depth`: Number of bits in a pixel.

`pixel_type`: Data type of the pixel.

`projection`: Projection for the image. Should be “Equirectangular” in all cases.

`projection_units`: Units used in the projection. Should be “meters” in all cases.

`samples`: Number of samples in the image.

`upper_left_easting_m`: Easting (in meters, see Section 3.9.1) of the upper left corner.

`upper_left_northing_m`: Northing of the upper left corner.

`x_scale`: X scale factor in meters per pixel.

`y_scale`: Y scale factor in meters per pixel.

`z_scale`: Z scale factor. Should be 1.0 because all DEM data are in meters.

5.4.2. Origin

The `orbital_origin.csv` file specifies where the origin of the map image is in relation to the reference frame with which it was created. In other words, the reference frame is whatever was entered by the user when the frame was created (contrast this with the next section). This preserves the original data, before any coordinate transforms have been applied.

The format is a CSV file (one line per orbital frame), but the semantic contents are the same as the localization raw files described in Section 5.1.4, so the description is not repeated here (the label has a short description of each field). The important values are the offset and the reference frame for the offset. The offset is the location of the point defined by `origin_line` and `origin_sample` in the metadata.

An example file is below.

```
frame,index,solution_id,x,y,z,offset_from,offset_from_date,offset_ref,offset_ref_date,
offset_ref_view,quat_s,quat_v1,quat_v2,quat_v3,quat_from,quat_from_date,quat_ref,
quat_ref_date,quat_ref_view,institution,creation_date,add_date,method,user

ORBITAL,0,telemetry,0.0,0.0,0.0,ROOT(0),2021-02-18 01:15:15.0,ROOT(0),2021-02-18 01:15:15.0,
telemetry,1.0,0.0,0.0,0.0,ROOT(0),2021-02-18 01:15:15.0,ROOT(0),2021-02-18 01:15:15.0,
telemetry,JPL,2021-02-18,2021-02-18,Telemetry,kgrimes

ORBITAL,1,telemetry,1099000.0,4345000.0,0.0,ORBITAL(0),2021-02-18 01:15:15.0,ORBITAL(0),
2021-02-18 01:15:15.0,telemetry,1.0,0.0,0.0,0.0,ORBITAL(0),2021-02-18 01:15:15.0,ORBITAL(0),
2021-02-18 01:15:15.0,telemetry,JPL,2021-02-18,2021-02-18,Telemetry,kgrimes

ORBITAL,2,telemetry,1099000.0,4345000.0,0.0,ORBITAL(0),2021-02-18 01:15:15.0,ORBITAL(0),
2021-02-18 01:15:15.0,telemetry,1.0,0.0,0.0,0.0,ORBITAL(0),2021-02-18 01:15:15.0,ORBITAL(0),
2021-02-18 01:15:15.0,telemetry,JPL,2021-02-18,2021-02-18,Telemetry,kgrimes

ORBITAL,3,telemetry,1093410.008,4354384.98,0.0,ORBITAL(0),2021-02-18 01:15:15.0,ORBITAL(0),
2021-02-18 01:15:15.0,telemetry,1.0,0.0,0.0,0.0,ORBITAL(0),2021-02-18 01:15:15.0,ORBITAL(0),
2021-02-18 01:15:15.0,telemetry,JPL,2021-02-18,2021-02-18,Telemetry,kgrimes
```

5.4.3. Global Origin

The `orbital_origin_global.csv` file specifies where the origin of the map image is in relation to the global coordinate system. In other words, the reference frame is orbital(0), regardless of what was entered by the user when the frame was created (contrast with the previous section). The common origin makes this file more generally useful than the (non-global) origin file (although in all cases to date, the reference frame is ORBITAL(0), so in practice there's no difference between these files).

The format is a CSV file with far fewer fields than the `orbital_origin.csv` file (because reference frames do not need to be defined). Again, they are not described in detail here, but short descriptions are in the label.

An example file is below.

```
frame,index,view,ref_frame,date_added,x,y,z
ORBITAL,0,telemetry,ORBITAL(0),2021-02-18 01:15:15.0,0.0,0.0,0.0
ORBITAL,1,telemetry,ORBITAL(0),2021-02-18 01:15:15.0,1099000.0,4345000.0,0.0
```

```
ORBITAL,2,telemetry,ORBITAL(0),2021-02-18 01:15:15.0,1099000.0,4345000.0,0.0
ORBITAL,3,telemetry,ORBITAL(0),2021-02-18 01:15:15.0,1093410.008,4354384.98,0.0
```

5.5. Collection: data_mechanisms

The primary purpose of PLACES is to store localization data. However, it also stores mechanism information – mostly joint angles for articulated mechanisms. Unlike localizations, there are no solutions for mechanisms. The mechanism values are assumed to be known, with no room for differing opinions. While this is not always the case in practice (for example, mast pointing is often adjusted during mosaic processing), such updates are not captured in PLACES.

Each mechanism's data are contained in a single CSV file in the `data_mechanisms` directory. These files are:

- `mech_arm.csv`
- `mech_drill.csv`
- `mech_drive.csv`
- `mech_hga.csv`
- `mech_rsm.csv`
- `mech_sha.csv`

Note that other mechanisms exist, but are not currently supported in PLACES (for example, suspension and the bit carousel). These may be supported in future releases.

5.5.1. Sources of Information

Mechanism information is collected from various places, depending on the mechanism. Each one's source data are described in the appropriate section. However, there are two sources of information that are common to most mechanisms; they are described below.

This document is not intended to document the information sources in detail. The information is provided in order to better understand the data, for those who are familiar with the telemetry data. Understanding the data source is not critical to using these data.

5.5.1.1. Telemetry Common Header

Every M20 data product sent via the telemetry mechanism has a header associated with it. This common header is contained in the "`.emd`" file in telemetry and contains a snapshot of the rover state at the time the data product was created. PLACES monitors the header for all telemetry products (with a few exceptions for those with known problems) and enters all the information into PLACES.

It should be noted that rover locations and orientations (for the "telemetry" view) are also extracted from this common header.

5.5.1.2. Image Labels

Every image product contains metadata labels, as described in the Camera SIS [1]. These labels (except for the EDL cameras and the helicopter) contain a snapshot of the rover state at the time the image was taken (which is not necessarily the same as the time the data product was created,

thus the need to look at both the image labels and the telemetry common header). The available data in this snapshot varies per instrument and mode; see the Camera SIS for details.

PLACES monitors all images and ingests the information it finds. Rover locations and orientations (for the "telemetry" view) are also extracted from this common header.

5.5.1.3. Specific Data Products

Other mechanisms get information by monitoring data products specific to the mechanism, or a general rover kinematics state product (called `MotRks`). These are listed in the individual mechanisms' descriptions.

5.5.2. Common Format

All mechanism data files are in a CSV format. The first several fields are common across all mechanisms; only the last few (containing the actual mechanism info) vary. For example, the arm mechanism file looks like this:

```
frame,site,drive,hga,min_sclk,max_sclk,approx_min_sclk,approx_max_sclk,azimuth,elevation
INMOTION,1,0,2,666952977.663,666952977.663,666952977.663,666952977.663,0,-2.22619e-05
INMOTION,1,44,2,666952977.663,666956424,666952977.663,666956424,0,-2.22619e-05
INMOTION,1,52,2,666957081,666968221,666957081,666968221,0,-2.22619e-05
INMOTION,1,52,94,667035458.958,667132185.035,667035458.958,667132185.035,1.13454,-0.784986
INMOTION,1,52,178,667217031.042,667222089.066,667217031.042,667222089.066,1.13453,-0.784986
INMOTION,1,52,262,667300843.053,667310972.991,667300843.053,667310972.991,1.13454,-0.784986
INMOTION,2,0,188,667741036,667750176,667741036,667750176,1.13455,-0.784986
INMOTION,2,0,198,667750299,667751507,667750299,667751507,1.13452,-0.78502
INMOTION,3,0,0,667751508,668009803.736,667751508,668014223,1.13452,-0.78502
INMOTION,3,0,102,667818885,667905416,667818885,667905416,1.13455,-0.784986
INMOTION,3,0,204,667907657,667994190,667907657,667994190,1.13454,-0.784986
INMOTION,3,0,306,667996432,668082964,667996432,668082964,1.13453,-0.784997
...
```

The first 8 fields are common across all mechanism files; these are described below:

`frame`: Always INMOTION for mechanism files, should be ignored.

`site`: RMC Site component at the time this entry was valid.

`drive`: RMC Drive component at the time this entry was valid.

`<component>`: The name of this field varies per mechanism type. It is the RMC component that is relevant for the mechanism (the one that increments when the mechanism moves). It indicates the value of this component at the time this entry was valid. In the example above, the component is "hga".

`min_sclk`: Minimum SCLK for this entry (see below).

`max_sclk`: Maximum SCLK for this entry (see below).

`approx_min_sclk`: Approximate minimum SCLK (see below).

`approx_max_sclk`: Approximate maximum SCLK (see below).

In the above, the min and max sclk define the exact range over which it is known that the mechanism had this value. If they are the same, the mechanism could have been in motion, with

that value for only an instant in time (the corresponding mechanism-specific RMC element should be odd if it was in motion). Or it could be that PLACES had no information other than a specific instant in time. If the values are not the same, then it was known that the mechanism was in the same position for the entire time period. It is possible for the min and max sclk values to be null ("") or missing, if no exact values are known.

The approximate SCLKs exist because one of the major sources of information (telemetry common header) reports SCLK values only as integer seconds. Subseconds are not available. Thus, the approximate SCLKs are often rounded to whole integers. Unlike the non-approximated values, approximated values will always exist (they will often be identical to the non-approximated values, if the telemetry common header added no information).

5.5.3. Mechanism Values and Meanings

Note that within a given mechanism element, not all possible attributes will be present, depending on what was available in the source data. Generally, these attributes will be flagged with the value -9999, but they may also simply be absent. Internally, values of `1.0e+30` flag missing values, but these should not appear in the PDS output.

All mechanism values are in radians unless specified otherwise.

It is generally beyond the scope of this document to describe the exact meaning of these mechanism values – what exactly is being described, what 0 means, and what rotation direction is positive. This information should be available in the PPPCS ([3]), or in other Mars 2020 project documentation. For user convenience, parts of the PPPCS have been extracted and are included in this document; see Appendix B.

5.5.4. Resolvers vs. Encoders

Several of the mechanism angles are measured by two different sensors. The "encoder" is on the input side of the rotating shaft, attached to the motor. The "resolver" is on the output side of the shaft, attached to the moving mechanism. Thus, the encoder records motor movements, while the resolver records how the device actually moved (including the effects of backlash). For most purposes, the resolver is more accurate (because it takes backlash into account) and should be used if available. It is possible for the resolver data to be unavailable (say, in the event of hardware failure), in which case the encoder should be used instead.

5.5.5. Arm

The state of the rover's robotic arm is described in the `mech_arm.csv` file. The mechanism-specific fields are:

- `enc_azimuth`
- `enc_elevation`
- `enc_elbow`
- `enc_wrist`
- `enc_turret`
- `res_azimuth`

- res_elevation
- res_elbow
- res_wrist
- res_turret

The five joints are numbered, from the rover out, in the order (azimuth, elevation, elbow, wrist, turret). Both encoder and resolver values are available.

In addition to the telemetry header and image labels, arm information comes from the MotRks and ArmSummary telemetry products.

5.5.6. Drill

The state of the drill mechanism on the robotic arm turret is described in the mech_drill.csv file. The mechanism-specific fields are:

- chuck
- feed
- rotate

Drill data come only from the MotRks telemetry product.

5.5.7. Drive

The state of the rover wheels and steering is described in the mech_drive.csv file. The mechanism-specific fields are:

- steer_LF
- steer_LR
- steer_RF
- steer_RR
- drive_LF
- drive_LM
- drive_LR
- drive_RF
- drive_RM
- drive_RR

The drive_ fields describe the rotation of the wheels. These are not yet implemented. The steer_ attributes describe the steering of the wheels (and are implemented). The wheels are named by (Left|Right) (Front|Middle|Rear).

Drive data come from the MobSummaryReport and MotRks telemetry data products, in addition to the image labels.

5.5.8. HGA

The state of the HGA (High Gain Antenna) mechanism on the rover is described in the mech_hga.csv file. The mechanism-specific fields are:

- azimuth
- elevation

HGA data come from the HgaDpSummary and MotRks telemetry data products, in addition to the image labels.

5.5.9. RSM

The state of the RSM (Remote Sensing Mast) mechanism (camera head) on the rover mast is described in the mech_rsm.csv file. The mechanism-specific fields are:

- enc_azimuth
- enc_elevation
- res_azimuth
- res_elevation

RSM azimuth and elevation are raw joint angles. For elevation, it is in the range of 0 to 182 degrees (but measured in radians). 0 is against the lower hardstop, which is pointing one degree past straight down, and positive is up. Pointing level to the Rover Nav frame is thus nominally at elevation 91 degrees. Azimuth is in the range 0 to 362 degrees (again measured in radians). 0 is at the leftmost hardstop (looking one degree past the back of the rover), increasing to the right. Pointing straight ahead in the Rover Nav frame is thus nominally at azimuth 181 degrees.

Calibration data show that the straight ahead and level position is at azimuth 3.161115 radians (181.119 degrees), elevation 1.588138 radians (90.9936 degrees).

The above information is provided for convenience only, because of the importance of the RSM to many instrument observations. The PPPCS [3] should be considered the definitive reference, and any discrepancies should be resolved in favor of the PPPCS.

RSM data come from the RsmSummary and MotRks telemetry data products, in addition to the image labels and telemetry common header.

5.5.10. SHA

The state of the SHA (Sample Handling Arm) mechanism on the rover is described in the mech_sha.csv file. The mechanism-specific fields are:

- TBD in a future release

SHA data come from the MotRks telemetry data products, in addition to the image labels.

5.6. Collection: data_ancillary

There are two ancillary tables, managed by PLACES, that provide additional information. These tables are in the data_ancillary collection and are:

- observations.csv
- sclk_table.csv

These are described individually below. Note that PLACES also provides an "instruments" table, but because that merely lists the constant instrument names, it is not included in PDS. For those familiar with the MSL delivery of PLACES, the "saved frame" table is not used on Mars 2020.

5.6.1. Observations table

This is a table, `observations.xml`, that lists exposure times for all of the camera observations seen by PLACES, referenced to the SCLK at which the exposure was taken. This table was created in response to a very specific request from a customer and may or may not be generally useful. Note that Mastcam-Z and SHERLOC "recovered" products ("ECR" type) are not included in this table, nor are the EDL cameras or the helicopter cameras.

The SCLK is taken from the IDENTIFICATION.SPACECRAFT_CLOCK_START_COUNT label item. However, this is done during operations, not during PDS reprocessing. Early in the mission, the label item for Mastcam-Z was set to the Mini-Header time. This was changed to the Command Dispatch time for most Mastcam-Z images around sol 55. The image data in PDS has all been reprocessed, so the SCLK is correctly the Command Dispatch time in PDS. However, in these cases, the SCLK in the observations table will match the Mini-Header SCLK rather than the one in IDENTIFICATION.

```
sclk,inst,duration
666952977.663,FRONT_HAZCAM_LEFT_A,0.015161
666952977.663,REAR_HAZCAM_RIGHT,0.016511
666952984.226,FRONT_HAZCAM_LEFT_A,0.015111
666952984.226,REAR_HAZCAM_RIGHT,0.016311
666952990.226,FRONT_HAZCAM_LEFT_A,0.015111
666952990.226,REAR_HAZCAM_RIGHT,0.016311
...
```

The fields are:

`sclk`: Value of SCLK (spacecraft clock) when the image was taken.

`inst`: Which instrument acquired the image.

`duration`: Duration of the exposure, in seconds.

5.6.2. SCLK Table

This table, `sclk_table.xml`, contains the SCLK (spacecraft clock) ranges for every RMC seen by PLACES. It is perhaps the most useful of the ancillary tables.

A "static" (or stable) RMC is one for which all (relevant) indices (omitting Site) are even (see Section 2.2). This indicates the rover is not moving. A dynamic RMC (where the index is odd) indicates that the rover, or some mechanism on the rover, was moving. This is indicated by a "INMOTION" for the frame name.

Both are stored in the SCLK table. The minimum and maximum SCLKs seen for that static segment, or that in-motion segment, are recorded.

For static RMCs, the time period indicates the period during which the rover was **known** to be static. It does ***not*** mean the rover stopped moving just before the time period or started again just after. It could very well have been quiescent for quite some time before or after the indicated times, and we simply don't have the data to tell. We just know it was not moving during that time period. Similarly, a range of times for a dynamic RMC means it was moving during that time, but not that it was static outside the time. Bottom line, there are gaps in the SCLKs during which we do not know, based on available telemetry, what the RMC was.

The SCLK table is pulled together from many different sources of information (basically, all the sources described in Section 5.5.1). Many of these data sources do not provide a full RMC. For example, the HGA motion report contains the site, drive, and HGA indices, but not the rest. Thus, there are many entries in the SCLK table with -1 for various (mechanism, generally) RMCs. The given times apply just to the HGA motion; it is known for example that the HGA was static during the given interval if the HGA index is provided, but it is not known that the RSM was static if its index is -1. Each entry applies only to the RMC elements provided; it says nothing about when a mechanism was static if the corresponding RMC index is -1. This creates a lot of overlap in the ranges. For example, the HGA might be known to be static for the entire duration of an arm activity.

Because the SCLK table comes from so many different sources, the possibility exists for small inconsistencies in the table, with overlapping SCLK ranges. These overlaps should generally be less than a second.

An example of the SCLK table is provided below.

```
frame,site,drive,pose,arm,sha,drill,rsm,hga,bitcar,seal,min_sclk,max_sclk,min_approx_sclk,
max_approx_sclk
ROVER,3,386,8,0,0,0,58,102,0,0,668363757,668364940,668363757,668364940
ROVER,3,386,8,0,0,0,62,102,0,0,668365213,668366587,668365213,668366587
INMOTION,3,389,1,0,0,0,0,0,0,0,668365978,668365983,668365978,668365983
ROVER,3,392,2,0,0,0,0,0,0,0,668365999,668365999,668365999,668365999
INMOTION,3,392,3,0,0,0,0,0,0,0,668366003,668366003,668366003,668366003
INMOTION,3,393,1,0,0,0,0,0,0,0,668366003,668366003,668366003,668366003
INMOTION,3,397,1,0,0,0,0,0,0,0,668366012,668366015,668366012,668366015
ROVER,3,398,2,0,0,0,0,0,0,0,668366038,668366038,668366038,668366038
INMOTION,3,403,1,0,0,0,0,0,0,0,668366041,668366064,668366041,668366064
ROVER,3,404,2,0,0,0,0,0,0,0,668366067,668366067,668366067,668366067
INMOTION,3,404,3,0,0,0,0,0,0,0,668366067,668366067,668366067,668366067
ROVER,3,410,2,0,0,0,0,0,0,0,668366095,668366095,668366095,668366095
```

`frame`: Indicates INMOTION or static (ROVER) entry.
`site, drive, pose, arm, sha, drill, rsm, hga, bitcar, seal`: RMC components for this sclk entry.
`min_sclk`: Minimum sclk (see below).
`max_sclk`: Maximum sclk (see below).
`min_approx_sclk`: Approximate minimum sclk.
`max_approx_sclk`: Approximate maximum sclk.

In the above, the min and max sclk define the exact range over which it is known that the RMC was valid. It is possible for the min and max sclk values to be null ("") or missing, if no exact values are known.

The approximate SCLKs exist because one of the major sources of information (telemetry common header) reports SCLK values only as integer seconds. Subseconds are not available. Thus, the approximate SCLKs are often rounded to whole integers. Unlike the non-approximated values, approximated values will always exist (they will often be identical to the non-approximated values, if the telemetry common header added no information).

5.7. Collection: document

The `document` collection contains all the documents relevant for Rover PLACES, and labels for each.

`PLACES_M20_PDS_SIS.pdf`: this document

`Mars2020_Rover_PLACES_Labels_sort_pds.pdf`: see Appendix A

`Mars2020_Rover_PLACES_Labels_sort_pds.html`: see Appendix A

`Mars2020_Rover_PLACES_Labels_sort_vicar.pdf`: see Appendix A

`Mars2020_Rover_PLACES_Labels_sort_vicar.html`: see Appendix A

`PLACES_SIS_Appendix_B_PPPCS`: see Appendix B

`release_notes.txt`: Notes and issues that occurred too late to be incorporated into the documents

Note that the Camera SIS [1] is included by reference as a secondary element in the document collection. This means it is logically a part of the Rover PLACES delivery, but physically lives in the mars2020_mission bundle.

6. Glossary

areoid: Shape defining "sea level" on Mars, taking into account local gravity variations. Same as geoid on Earth.

attitude: Synonymous with "orientation".

chain: A set of links between Derived-from frames that implement the system by which one localization can affect other downstream ones.

clone: A copy of a view; shares the same parent but has a copy of the contents.

closed view: A view that cannot be modified. Views can be closed up to a certain point, identified with an RMC value.

coordinate frame: As used in this document, a method by which Cartesian (XYZ) coordinates can be expressed. When used alone, refers to a type of frame, such as Rover or Site. With an index, refers to a specific instance of that frame type, valid at some point in time, with a specific origin and orientation (with respect to some other frame).

data product: A file sent by the rover containing raw telemetry information of a certain type. Also, a labeled file that is part of the PDS archive.

deep copy: An expensive copy of a view, which may include several levels of parents.

derived-from frame: The frame from which a solution was derived. Need not be the same as the Reference frame.

drive: Second index of the RMC; indicates when the rover is driving (odd) or stationary (even).

dynamic RMC: An RMC with an odd value for Drive or Pose; indicates the rover is in motion or updating pose knowledge. Can also refer to odd values of mechanism RMC values, indicating the corresponding mechanism is in motion.

easting: Global-frame measurement of coordinates; indicates meters east of the prime meridian, as measured at the standard parallel.

estimate: Synonymous with "solution".

frame: An instance of a coordinate system. Combines the type of frame (Site, Rover, Local_Level, Orbital, root) with an instance index (RMC, SCLK, or counter). The frame has a single but unknowable true pose; solutions provide estimates of that true pose.

global frame: A coordinate frame used for mapping, with its origin at the equator and prime meridian. Global frame coordinates are directly translatable to latitude/longitude. The global frame is an instance of an orbital frame, named by ORBITAL(0).

groups: A mechanism to control access to views and solution IDs.

localization: The process of determining where the rover is and how it is oriented at any given time.

local level frame: A combination of Rover frame position with Site frame orientation.

mechanism: Any device on the rover that can move, such as wheels, arms, covers, or masts.

mechanism RMC: The last seven values of RMC (Arm, SHA, Drill, RSM, HGA, BITCAR, SEAL), which indicate motions of mechanisms.

metadata: Descriptive information attached to database entries (views, solution IDs, etc) that plays no part in coordinate transformations.

mobility RMC: The first three values of RMC (Site, Drive, Pose), which have special meaning for mobility and the PLACES database.

northing: Global-frame measurement of coordinates; indicates meters north of the equator.

open view: A view that can be modified. See closed view.

orientation: How the rover or a frame is oriented in space. Specifically, a rotation relative to the reference frame.

orbital frame: Any of several coordinate frames describing maps or map-like entities, distinguished by an index. The global frame is an instance of an orbital frame (index 0).

orbital images: Maps used as an aid to localization and areoreferencing (known as georeferencing on Earth).

PLACES: Database storing localization solutions for all operations users across the Mars 2020 project.

pose: The combination of position and orientation of the rover or a frame. When capitalized, indicates the third index of the RMC.

position: Where the rover or a frame is located. Specifically, a translation or offset in Cartesian XYZ space from the reference frame.

quaternion: A 4-element vector used to represent a rotation in 3-dimensional space. Used to represent orientation of the rover in PLACES.

reference frame: The frame in which coordinates (offsets or rotations) are expressed. See also Derived-from frame.

ReST web service: A mechanism for implementing web services using simple URLs and XML files.

root: The frame at the root of the coordinate frame tree, from which all others are derived. Defined to be the same as the global frame origin.

rover (navigation) frame: A coordinate frame attached to the rover, with +X pointing forward and +Z toward the belly, with +Y completing the right-handed system.

RMC (Rover Motion Counter): A set of 10 indices used as a clock to track rover motion. Used as the instance identifier for most Rover frames. The first three are the mobility RMC, the rest are the mechanism RMC.

saved frame: A mechanism used onboard to save and reuse some frames after the rover moves from that location. Not used on Mars 2020.

site: An area of operations; establishes a coordinate system origin for use in local operations.

site frame: A coordinate frame attached to the ground, with +X pointing north and +Z down according to the local gravity vector. Coincident with local_level at the time it is defined.

sol: One Martian day.

solution: One specific position or orientation (out of possibly many) for a given RMC. Represents an individual estimate of the unknown true value of position or orientation.

solution ID: A string used to identify a set of solutions.

SPICE: A toolkit to compute ancillary navigation-related data for space missions.

stable RMC: An RMC with even values for the Drive and Pose indices; indicates the rover is not intentionally moving or updating its pose knowledge. Can also apply to mechanism RMCs where an even value indicates the mechanism is not in motion.

subframe: A database implementation detail; combines dynamic RMC and unsolicited move frames in a single term.

telemetry: View containing only values obtained from the rover; it is the root of the view tree.

unit vector: A 3-element vector with magnitude 1.0, used to indicate a direction in space (such as the direction a camera is pointing) independent of position.

unsolicited move: An unexpected motion of the rover, such as slipping off a rock due to a wind gust.

view: A named entity which provides a single solution for each RMC. Used to differentiate different sets of solutions.

visual odometry (visodom): A mechanism to refine pose onboard by analyzing images before and after a drive.

XYZ: Cartesian coordinates used to express an offset or position.

7. Acronym List

BITCAR: Bit Carousel

CSV: Comma Separated Value

CTX: Context Camera (MRO)

DEM: Digital Elevation Model

DRT: Dust Removal Tool

EDL: Entry, Descent, and Landing

EDR: Experiment Data Record

EMD: Earth Metadata file (".emd")

ESRI: Environmental Systems Research Institute (software company)

EVR: Event Verification Record

FSW: Flight Software

GDAL: Geospatial Data Abstraction Library

GIS: Geographic Information System

GeoTIFF: Geographic Tagged Image File Format

HiRISE: High Resolution Imaging Science Experiment (on MRO)

HGA: High Gain Antenna

HRSC: High Resolution Stereo Camera (on Mars Express)

HTTP: HyperText Transfer Protocol

Hz: Hertz

IAU: International Astronomical Union

ID: Identifier

IDPH: Image (or Instrument) Data Product Header

IDS: Instrument Data Subsystem

IMU: Inertial Measurement Unit

JPL: Jet Propulsion Laboratory

LDAP: Lightweight Directory Access Protocol

Mastcam-Z: Mast Camera - Zoom

MER: Mars Exploration Rover

MIPL: Multimission Image (or Instrument) Processing Laboratory

MMM: Mastcam, MAHLI, MARDI

MOLA: Mars Observer Laser Altimeter

MRO: Mars Reconnaissance Orbiter

MSL: Mars Science Laboratory

M2020: Mars 2020 Rover (or Mission)

Navcam: Navigation Camera (Rover)

ODL: Object Description Language

OPGS: Operations Product Generation Subsystem

PDS: Planetary Data System

PDS4: Planetary Data System, Version 4

PIXL: Planetary Instrument for X-ray Lithochemistry

PLACES: Position Localization and Attitude Correction Estimate Storage

PNG: Portable Network Graphics

PPPCS: Pointing, Positioning, Phasing, and Coordinate Systems (document)

RDR: Reduced Data Record

ReST: Representational State Transfer

RMC: Rover Motion Counter

RSM: Remote Sensing Mast

RTT: Round Trip Tracking

SIS: Software Interface Specification

SCLK: Spacecraft Clock

SHA: Sample Handling Arm

SHERLOC: Scanning Habitable Environments with Raman and Luminescence for Organics and Chemicals

SPICE: Spacecraft, Planet, Instrument, Camera, Events

UHF: Ultra High Frequency

URL: Uniform Resource Locator

USGS: United States Geological Survey

VICAR: Video Image Communication and Retrieval

visodom: Visual Odometry

XML: eXtensible Markup Language

XYZ: Cartesian coordinates (not an acronym)

8. References

[1] Ruoff, Nicholas; Deen, Robert; and Pariser, Oleg, Mars 2020 Project Software Interface Specification, "Camera Data Products", JPL D-99960, Jan 12, 2022 (or as updated). urn:nasa:pds:mars2020_mission:document_camera:mars2020_camera_sis.pdf https://pds-geosciences.wustl.edu/m2020/urn-nasa-pds-mars2020_mission/document_camera/Mars2020_Camera_SIS.pdf

[2] Deen, Robert, Mars Science Laboratory, "PLACES User Guide", MSL-586-3653, JPL D-71121, Aug 19, 2011 (or as updated).

[3] Walker, Luke; Alibay, Farah; Williams, Jeffrey B; et al, "Pointing, Positioning, Phasing, & Coordinate Systems (PPPCS)", collection of Mars 2020 Flight System documents, Document Numbers 97208, 97216, 97212, 97213. Various dates, 2020.

[4] Smith, D.E.; Zuber, M.T.; Frey, H.V.; Garvin, J.B.; Head, J.W.; Muhleman, D.O.; Pettengill, G.H.; Phillips, R.J.; Solomon, S.C.; Zwally, H.J.; Banerdt, W.B.; Duxbury, T.C.; Golombek, M.P.; Lemoine, F.G.; Neumann, G.A.; Rowlands,D.D.; Aharonson, Oded; Ford, P.G.; Ivanov, A.B.; Johnson, C.L.; McGovern, P.J.; Abshire, J.B.; Afzal, R.S.; and Sun, Xiaoli, "2001, Mars Orbiter Laser Altimeter—Experiment summary after the first year of global mapping of Mars" *Journal of Geophysical Research*, v. 106, no. E10, p. 23,689–23,722.

[5] Planetary Data System Standards Reference, JPL D-7669, Part 2, version 1.16.0.0, April 21, 2021, https://pds.nasa.gov/datastandards/documents/sr/.

[6] Planetary Data System Data Provider's Handbook, version 1.16.0.0, April 21, 2021, https://pds.nasa.gov/datastandards/documents/dph/.

[7] PDS4 Information Model Specification, version 1.16.0.0, April 2021, https://pds.nasa.gov/datastandards/documents/im/.

PDS4 is the name of the current PDS archive standard, described in the above documents. The PDS4 Information Model, on which these documents are based, is revised about every six months, and the documents are revised accordingly. The most recent versions may be found at https://pds.nasa.gov/datastandards/documents/. The PDS4 products in the PLACES archive have been designed based on the versions current at the time, which are those listed above.

9. Change Log

This is not intended to be an exhaustive change log, but rather an overview summary of changes in the document.

Version 1.0 (PDS Release 3)

- Initial version, derived from the MSL PLACES document.

Appendix A: Product Label Keyword Definitions

This Appendix consists of a set of tables that describe the PDS4 keywords (classes and attributes in PDS 4 parlance) used in all labels in the `mars2020_rover_places` bundle. They include pointers to the matching VICAR keywords (where applicable, just for the `data_maps` collection), as well as both the generic (multimission) definition of the keyword, and the specific "nuance" or supplemental information that applies only to Mars 2020.

The tables are separate files (referenced below) rather than being incorporated inline in the document.

These tables are created by examining a set of sample labels (incorporating all types of products being created) in order to determine the PDS4 classes and attributes that are actually used by the products described in this SIS. This list is then cross-referenced against the PDS4 data dictionaries in order to find the definitions, children, valid values, and data types. This list is then augmented with "property maps" that provide the Mars 2020-specific valid values, and the "nuance" definitions.

These tables are thus much more useful for most purposes than looking at the PDS4 data dictionaries directly, because they contain *only* the keywords that are *actually* used in the bundle.

The first column contains the name of the PDS4 attribute (keyword) or class (container), and the PDS4 dictionary it comes from. Along with that, when applicable, are the VICAR keyword and property name(s) from which the values are derived. The property name is the section of the VICAR label. Not every entry has a VICAR keyword; some entries are merely containers, others contain constants or values that are derived in other ways. Some of the VICAR keywords refer to the class rather than the attribute; for example a VICAR vector keyword will typically refer to the vector's class rather than the x,y,z attributes individually.

The second column contains the definition. There are two components to many definitions, as alluded to above. The first, which is always present, is the standard PDS4 definition that applies to all missions, from the PDS4 data dictionary. The second (in italics) is a Mars 2020-specific nuance to the definition, providing additional context that applies specifically to Mars 2020.

The third column is broken up into several pieces. The first is the XPath. This gives the "path" of where the item can be found in the label, tracing the hierarchy from the root (often but not always Product_Observational) down to the item itself. Each level in this hierarchy is a hyperlink, which can be clicked on to go directly to that items' definition.

Underneath the XPath is a field whose content varies based on the type. For attributes ("keyword"), this contains the valid values, when such are defined either by the PDS4 data dictionary or the Mars 2020-specific property maps. For classes (containers), the valid children are listed. Those that are blue hyperlinks are actually used by Mars 2020 Rover PLACES; clicking on them will go to that item's definition. Those that are not blue are defined by the PDS4 data dictionary but are not used by Mars 2020 Rover PLACES.

Finally, also underneath the XPath field is another column containing (for attributes only) the data type and units. All attributes should have a data type, but only some have units defined.

These label tables are the primary source of information regarding the metadata in the labels. The rest of this document describes things at a high level; the label tables (along with the ops label table, above) define specifically what each label item means.

There are two versions of the table: sorted by PDS4 name, and sorted by VICAR keyword. The tables can be used in either direction. Given a label item you don't understand, you can look it up in the table (sorted by either PDS or VICAR name, depending on which you're looking at) to find the definition. In some cases you may need to go up the hierarchy to find a meaningful definition (for example the definition for "x" is not particularly useful, but the parent or grandparent should describe what the full x,y,z value is being used for). Alternatively, given an item in the table, you can find the item in the label by following the XPath – looking down the hierarchy of elements until you find the item. Note that not all keywords are in any given label; the table encompasses all labels in the bundle: map images, CSV files, XML files, documentation files, and the PDS4 structural labels (such as bundle and collection labels).

The cross-reference between PDS4 name and VICAR keyword can also be useful for comparing values across similar missions (MSL, MER, Phoenix, etc) that use PDS3 (the VICAR and PDS3 keywords are generally the same).

Each of the tables is provided in both HTML and PDF format. The files are:

`Mars2020_Rover_PLACES_Camera_SIS_Labels_sort_pds.pdf`

`Mars2020_Rover_PLACES_Camera_SIS_Labels_sort_pds.html`

`Mars2020_Rover_PLACES_Camera_SIS_Labels_sort_vicar.pdf`

`Mars2020_Rover_PLACES_Camera_SIS_Labels_sort_vicar.html`

Appendix B: Subset of PPPCS

This Appendix to the Rover PLACES SIS is provided as a convenience to the user. It is an extraction, with minimal reformatting, of parts of the PPPCS (Pointing, Positioning, Phasing, & Coordinate Systems) document that are relevant to help understand the mechanism angles in PLACES. The entire document contains items that may not be releasable due to export restrictions or other concerns. This subset should be sufficient to understand the mechanisms.

Because it is an extraction from a much larger document, there may be acronyms or references that are not resolved in the text. The Rover PLACES SIS authors believe that in most cases these are either not particularly relevant, or are self-explanatory.

The Appendix is in a separate file in the documents collection:

`Mars2020_Rover_PLACES_SIS_Appendix_B_PPPCS.pdf`